ADDING PAYMENT TO YOUR EXPERIENCE

TABLE OF CONTENTS

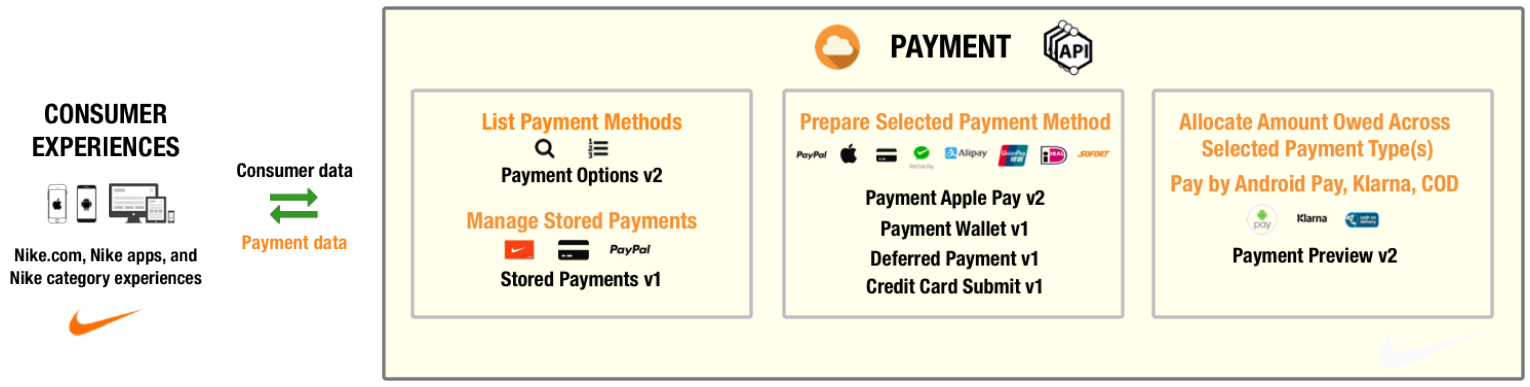
Adding Payment to Your Experience

Last Updated: 01/04/2023

Manage the payment process for consumers purchasing Nike products and services.

> **TIPS**:
>
> - Before using this guide, read Using Nike APIs and Payment Overview.
> - Use this Developer's Guide as a supplement to the API Reference for detailed use cases. See API Quick Reference for links to all the API Reference docs discussed in this guide.
> - The steps involving **Checkout** are covered in the Carts and Checkout guides.

Introduction



The payment process during Checkout consists of four steps:

1. Listing Payment Methods and Managing Stored Payments

Your experience can get a list of stored payments for a logged-in consumer by calling the Stored Payments service. The Stored Payments service can also be used to add, delete, and update a consumer's stored payments, including the default stored payment. The Payment Options service lists and validates non-stored payments.

2. Preparing Payment for Purchase

Depending upon the payment type, your experience needs to perform different actions to prepare the payment for purchase. Before a consumer can pay with Apple Pay, you must start an Apple Pay session. To allow consumers to pay in the PayPal Express or PayPal Mark flows, call the Wallet Payment service to start a PayPal session. When paying by a non-stored credit card, your experience collects the consumer's credit card information using the Credit Card Payment service. If consumers pay by a Deferred Payment type such as Alipay or WeChat, your experience generates a signed URL and redirects the consumer, so they can pay at the vendor's site after they submit the Nike Checkout. If your consumer is shopping in Korea, Request a Ready Payment with the appropriate vendor.

3. Payment Preview

Because consumers can pay for their Checkout using Gift Cards, Vouchers, and another payment type, it is necessary to calculate how much of the Checkout will be paid by each payment type by calling Payment Preview. Your experience can display the payment allocation results to consumers, so they can verify their payment details before submitting the Checkout.

4. Payment Approval

Before the Checkout can be submitted for fulfillment, payment information needs to be validated and certain payment types need to be authorized to make sure there are enough funds. Both validation and authorization are handled by Payment Approval, but your experience does not need to call the endpoint directly. The Checkout API does it for you when you call Request a Checkout Submit.

> **TIP**: See the Best Practices section for sample payment flows.

Notifying Nike of Payment After Checkout

When consumers pay with a deferred payment type, they pay for their order at a third-party vendor site after submitting the order for fulfillment. Because the payment event happens outside of the Nike Checkout flow, third-party vendors notify Nike of payment events through the Third-Party Payment Notification service.

Payment Status Changes During Fulfillment

After an Order has been submitted for fulfillment, it goes through a series of statuses, some of which involve payment. The Document Order Management System (DOMS) calls the Fulfillment Payment Notification service to request debits, credits, voids, re-authorizations, and to get payment status.

Payment APIs for Checkout v3

If you are integrating with the Checkout v3 APIs, you must use the latest endpoints for certain (but not all) Payment APIs. Below is a summary of these endpoints and what is different about them as compared to the prior versions.

Table 1: Payment Endpoints for Checkout v3

Endpoint Name	Request Differences	Response Differences
Payment Options v3	`fulfillmentDetails` object per item	None
Payment Wallet v2	Express flow: fulfillment totals. Mark flow: `fulfillmentDetails` per item	None
Payment Preview v3	`fulfillmentDetails` object per item	None
Payment Approval v3	`fulfillmentDetails` object per item, also fulfillment section in `totals`	None

More on `fulfillmentDetails`

Some Payment endpoints for use with Checkout v3 allow the optional inclusion of a `fulfillmentDetails` object in the request body. This object contains data you previously got from the Fulfillment Offerings API.

The `fulfillmentType`, `getBy` and `maxDate` values you send in the request may affect the response. As such, these API versions are sometimes referred to as ‘source-aware’, because they return different results depending on the source of fulfillment for each item in the checkout.

Key Terms

Table 2: Key Payment Terms

Term	Definition
3D Secure 1	Payment authentication where consumers leave the checkout flow to perform Strong Customer Authentication (SCA) at a bank site. Once authenticated, the consumer is returned to the shopping flow to complete checkout.
3D Secure 2	Frictionless payment authentication where consumers perform SCA within the shopping flow. SCA can be performed passively such as through an API that obtains a consumer’s device fingerprint or actively where a consumer completes an online challenge.
Authorization	A temporary hold on funds in a consumer’s account for a future charge
Credit	Funds that are returned to a consumer’s account
Debit	Funds that are removed from a consumer’s account
Deferred Payment	A type of payment where a consumer places an order and then pays for it at a Third-party bank
DOMS	A Distributed Order Management System, also known as Sterling, that handles order fulfillment

Term	Definition
ESB	Enterprise Service Bus, similar to PAC but used to communicate with Nike's non-commerce systems
PAC	Messaging system used by DOMS to communicate with other Nike commerce systems
PCI-DSS	Payment Card Industry Data Security Standard provides secure standards for handling credit card data. All Nike CiC payment services are PCI-DSS compliant.
Reauthorization	When a temporary hold on funds in a consumer's account is reissued, typically when the original authorization has expired
Ready Payment	All non-stored Korea payments must go through the Ready Payment process to gather information needed by a payment vendor when consumers go to the vendor's site to authenticate
S3	Amazon Simple Storage Service used to store and retrieve data such as files
Strong Customer Authentication	Process where consumers provide something they have (e.g. device fingerprint) and/or know (e.g. password) in order to be authenticated.
Void (of payment)	Reverses a successful payment authorization, also known as an authorization reversal

Supported Stored Payment Types

The Stored Payment Service supports storing these types of payment:

Table 3: Supported Stored Payment Types

Payment Type Description	Value	Storage Limit
Alipay	**AliPay**	1
Apple Pay	**ApplePay**	1
Credit Card	**CreditCard**	4
Gift Card	**GiftCard**	10
PayPal	**PayPal**	1
Tenpay	**TenPay**	1
UnionPay	**UnionPay**	1
WeChat	**WeChat**	1

Payment Options by Country

See Global Payment Options for a list of supported payment types by shipping and billing country.

Listing and Validating Payment Options

List payment options for Checkout

List billing countries for a shipping country

Validate payments

Step 1: List Payment Options for Checkout

Getting a list of valid payment options to display to your consumer is typically the first step in adding payment to your experience. This endpoint returns list of valid payment options based on the consumer's Nike UPMID, shopping country, billing country, currency, items, and in the case of v3, fulfillment details.

v2 Checkout

For v2 Checkout, use the Get Payment Options v2 endpoint:

```
GET https://api.nike.com/payment/options/v2
```

v3 Checkout

For v3 Checkout, use the Get Payment Options v3 endpoint:

```
GET https://api.nike.com/payment/options/v3
```

Common Considerations

- The results of a successful 200 response lists valid payment methods that a consumer can use to pay for the Nike checkout. The list includes the payment name (e.g. “Visa”) and payment type (e.g. “CreditCard”).
- When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
- Even though `items` is an optional request field, it is recommended that you pass it if available so product validation is performed as early as possible in the purchase flow.
- The endpoint validates all items passed in the request body. For performance reasons, the products are held in cache for 15 minutes. After the cache expires or if the product is not in cache, the service attempts to get fresh product data from the Merchandised Product API. If the Merchandised Product service is unreachable, the service defaults the product type to “INLINE” and continues validating the product.
- It is best practice to send all optional request headers, if the data is available, to avoid unexpected responses.

Step 2: List Billing Countries for a Shipping Country

You can get the list of valid billing countries based on the consumer’s shipping country using the Get Billing Countries for Shipping Country endpoint. Your experience can use this list to display only valid billing countries to the consumer and perform shipping/billing country validation as early in the purchase process as possible.

Note that the results are unsorted.

Listed below is a sample Get Billing Countries for Shipping Country GET request URI. It is not JWT-restricted.

```
https://api.nike.com/paymentoptions/options/v2/US
```

A successful 200 response lists the billing countries valid for the shipping country path parameter.

> **TIPS:**
>
> - The consumer’s billing country must be in the billing country results list in order for them to make a purchase.
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Your experience needs to pass the country code that the consumer is shopping in the `shippingCountry` query parameter.

Step 3: Validate Payments

Use the Validate Payments endpoint to check each payment on the Checkout selected by the consumer is valid for the shipping country. The POST request body should include a client-generated UUID, payment type and billing country for each Checkout payment, and the shipping country.

Listed below is a sample Validate Payments POST request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/validate_payments/v2
```

A successful 200 response lists all payments provided in the request and true if the billing country and payment type combination is valid, false if not.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - It is best practice to send all optional request headers and body fields, if the data is available, to avoid unexpected responses.

Storing Payment

Add, modify, delete, list stored payments

Modify the default stored payment

Validate a stored credit card

Start a PayPal billing agreement

Start and save a Fiserv billing key registration

The Stored Payment service is used to manage (add/update/delete/list) a consumer's stored payments. Consumers must be registered Nike members and log in to use stored payment. Guest consumers are not supported. See Supported Stored Payment Types to get storage limits by payment type.

Add a New Stored Payment

Use the Add Stored Payment endpoint to save a consumer's payment for future use. For PayPal stored payments, see Start a PayPal Billing Agreement. The request body varies depending upon the payment type and whether the endpoint is called pre-authorization or post-authorization.

The typical flow for storing a new credit card **pre-authorization** for a consumer is:

1. Call the Add Credit Card with CVV endpoint of the Credit Card Payment service to securely transmit credit card information via iFrame to the PCI-certified Credit Card Payment service, passing a client-generated UUID as the `creditCardInfoId`.
2. Call Add Stored Payment, passing the same `creditCardInfoId` from **Step 1** to look up the credit card information from short term storage and save it to long term storage.
3. Call Get Stored Payments by UPMID to get all stored payments including the one just saved to confirm that the credit card was securely stored. Credit Card account numbers are masked in the response.

   Listed below is the Add Stored Payment POST request URI. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

   ```
   https://api.nike.com/commerce/storedpayments/consumer/savepayment/
   ```

   A successful response is a 201.

Modify a Credit Card Stored Payment

Use the Modify Credit Card Stored Payment endpoint to update a stored credit card's expiration month, expiration year, billing address and default payment type flag, or to update a gift certificate default payment type flag. If you only need to update the default payment type flag of either a credit card or gift certificate stored payment, see Modify the Default Stored Payment.

The typical flow for modifying a stored credit card is:

1. Get Stored Payments by UPMID to get all of a consumer's stored payments. Credit Card account numbers are masked in the response.
2. Call Modify Credit Card Stored Payment . Get the `payment_id` path parameter from the appropriate `paymentId` field that was returned in the response of **Step 1**.
3. Get Stored Payments by UPMID to get all stored payments including the one just modified to confirm that the credit card was updated. Credit Card account numbers are masked in the response.

   Listed below is the Modify Credit Card Stored Payment PUT request URI. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

   ```
   https://api.nike.com/commerce/storedpayments/consumer/storedpayments/
   1686062b-4246-4c3b-ac96-e82a0efae7a0?includebalance=false
   ```

   A successful response is a 202.

Modify the Default Stored Payment

Use the Modify the Default Stored Payment' endpoint to update the default stored payment. This endpoint removes the default flag on the current default stored payment and adds the flag to the stored payment matching the `payment_id` path parameter. An experience can use the default payment type to pre-select a payment method in the shopping flow. If you need to update a stored credit card's expiration month, expiration year, billing address and default payment type flag, see Modify a credit card stored payment.

The typical flow for modifying the default stored payment is:

1. Get Stored Payments by UPMID to get all of a consumer's stored payments. Credit Card account numbers are masked in the response.
2. Call Modify Default Stored Payment passing the `payment_id` path parameter using the appropriate `paymentId` field returned in the response of **Step 1**.
3. Get Stored Payments by UPMID to get all stored payments including the one just modified to confirm that the default credit card was changed. Credit Card account numbers are masked in the response.

Listed below is a Modify Default Stored Payment PUT request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/commerce/storedpayments/consumer/storedpayments/
7661aea5d-31b6-4ac4-8830-1d61d2c7b043/default
```

A successful response is a 202.

Delete All Stored Payments

Use the Delete Stored Payments by UPMID to delete all of a consumer's stored payments. If you want to delete just one of a consumer's stored payments, see Delete Stored Payment by ID.

The typical flow for deleting all stored payments for a consumer is:

1. Get Stored Payments by UPMID to get all of a consumer's stored payments. Credit Card account numbers are masked in the response.
2. Call Delete Stored Payments by UPMID to delete all of a consumer's stored payments.
3. Get Stored Payments by UPMID to verify that no stored payments are returned.

Listed below is a sample Delete Stored Payments by UPMID DELETE request URI. **This endpoint is JWT-restricted.**

```
https://api.nike.com/commerce/storedpayments/consumer/storedpayments/
```

A successful response is a 204.

Delete Stored Payment by ID

Use the Delete Stored Payment by ID endpoint to delete a stored payment by payment ID. For example, this endpoint would be called when consumers delete a stored payment when managing their payment information in the experience.

The typical flow for deleting a stored payment for a consumer is:

1. Get Stored Payments by UPMID to get all of a consumer's stored payments. Credit Card account numbers are masked in the response.
2. Call Delete Stored Payment by ID to delete a consumer's stored payment for the given `payment_id`.
3. Get Stored Payments by UPMID to verify that no stored payments are returned.

Listed below is a Delete Stored Payment by ID DELETE request URI. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

```
https://api.nike.com/commerce/storedpayments/consumer/storedpayments/
37448493-6aea-4a9b-b250-742d3a26c081/?currency=usd
```

A successful response is a 204.

List Stored Payments

Get Stored Payments by UPMID

Use the Get Stored Payments by UPMID endpoint to list all of a consumer's stored payments. Account numbers are masked in the response. If you are a retail client, use the Get Stored Payments by UPMID (Retail) endpoint instead.

If the request does not contain a shipping address, or the shipping address sent does not match a shipping address in a previously placed order, the `validateCVV` field will be set to true in the response for Credit Card payment types. This flag indicates that the consumer must provide the CVV, and it must be validated before the consumer can pay for their order using the credit card in the checkout flow.

Listed below is a sample Get Stored Payments by UPMID POST URI request. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

```
https://api.nike.com/commerce/storedpayments/consumer/
storedpayments?currency=USD&includebalance=true&validateshipping=true
```

A successful 200 response lists all of a consumer's stored payments.

Get Stored Payments by UPMID Retail

Use the Get Stored Payments by UPMID Retail if you are a retail client to list all stored payments for a consumer. If you are not a retail client, use the Get Stored Payments by UPMID endpoint instead.

Listed below is a sample Get Stored Payments by UPMID (Retail). It is not JWT-restricted.

```
https://api.nike.com/commerce/storedpayments/consumer/retail_stored_payments/
v1?currency=USD
```

A successful 200 response lists all of a consumer's stored payments except credit cards that have not been used to place a Nike order.

Get Stored Gift Certificate by UPMID

Use the Get Stored Gift Certificate by ID endpoint to list details for a consumer's saved gift certificate.

Listed below is a sample Get Stored Gift Certificate by ID GET request URI. It is not JWT-restricted.

```
https://api.nike.com/commerce/storedpayments/consumer/giftcard/
79847893284923483924?currency=USD
```

A successful 200 response contains gift certificate details with a masked account number.

Get Stored Payment by ID

Use this endpoint to list stored payment details for a `payment_id`. This endpoint is primarily for gift cards, but it can be called for any type of stored payment.

When listing a gift card payment type, and you don't need the balance, pass `includebalance=false` as a URI parameter for a quicker response. Setting this parameter to false prevents the Stored Payments service from making a balance call to the gift card provider.

Listed below is a Get Stored Payment by ID GET request URI. **This endpoint is JWT-restricted**.

```
https://api.nike.com/commerce/storedpayments/consumer/storedpayments/
79847893284923483924
```

A successful 200 response varies depending upon the type of stored payment. A subset of response fields are listed below.

Credit Card

Instead of using credit card number as a lookup key, both the third-party payment gateway (TPPG) and Stored Payment use the `paymentToken` generated by the TPPG.

- **type**: always `CreditCard`
- **paymentToken**: Subscription id assigned to this payment by TPPG
- **cardType**: Type of credit card
- **expiryYear**: Year credit card expires, required for CreditCard type

- **expiryMonth**: Month credit card expires, required for CreditCard type

Gift Card

- **type**: always `GiftCard`
- **paymentToken**: Random UUID
- **accountNumber**: Unmasked account number
- **balance**: Gift card balance. Not returned if `includebalance` query parameter is false
- **pin**: Gift card PIN

PayPal

- **type**: always 'PayPal'
- **paymentToken**: PayPal billing agreement id
- **payer**: consumer's PayPal email address
- **payerId**: PayPal generated unique id

Deferred Payment

- **type**: Payment type
- **bankName**: Bank name for China payments

Validate Stored Payment Credit Card CVV

Use the Validate Stored Payment Credit Card CVV endpoint to validate a credit card or gift certificate based on the request shipping address. If the shipping address does not match a shipping address on a past order or is not sent, the response indicates that the CVV needs to be validated. This endpoint also validates if a credit card is expired. Note that billing address is returned.

Listed below is a sample Validate Stored Payment Credit Card CVV POST request URI. **This endpoint is JWT-restricted**.

```
https://api.nike.com/commerce/storedpayments/consumer/storedpayments/
438364275151500000l516?includebalance=false
```

A successful 200 response returns credit card or gift certificate validation and billing information for a `payment_id`.

Start a PayPal Billing Agreement

In order to save PayPal as a stored payment, the consumer must Start a PayPal Billing Agreement. This agreement pre-authorizes Nike to charge the consumer's PayPal account for purchases without requiring the consumer to visit the PayPal site to authorize each purchase. Once the consumer sets up a PayPal billing agreement, purchasing by PayPal is easy because the consumer can stay in your experience without having to go the PayPal site.

Follow these steps to allow consumers to Add a PayPal stored payment in your experience.

1. Call Get Stored Payments by UPMID to get a list all the consumer's saved payments to make sure the consumer doesn't already have a PayPal stored payment. Consumers can have only one PayPal stored payment.
2. Call the Start a PayPal Billing Agreement endpoint, passing the `returnURL` and `cancelURL` as query parameters so PayPal can return the consumer to your experience.
3. Redirect the consumer to the `redirectURL` in the response, so the consumer to provide payment details, approve, and subscribe to the Billing Agreement. Once the consumer accepts or cancels the Billing agreement, PayPal redirects the consumer to either the `returnURL` or `cancelURL` provided in the query parameter.
4. Call the Add Stored Payment endpoint to save the PayPal billing agreement.
5. (Optional) Call Get Stored Payments by UPMID to list all the consumer's saved payments.

   Listed below is a sample Start a PayPal Billing Agreement GET request URI for the SNKRS mobile experience. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

   ```
   https://api.nike.com/commerce/storedpayments/consumer/
   paypalagreement?returnUrl=http://nike.com/SNKRS/
   PayPalAuthenticationSucceeded&cancelUrl=http://nike.com/SNKRS/
   PayPalAuthenticationDidNotSucceed&design=mobile
   ```

   A successful 200 response lists the `requestToken`, `paypalToken`, and `redirectURL` at which the consumer can accept the billing agreement.

Start and Save a Fiserv Billing Key Registration

In order to save a Fiserv credit card as a stored payment, the consumer must Start a Fiserv Billing Key Registration. This agreement pre-authorizes Nike to charge the consumer's Fiserv credit card for purchases. Storing their Fiserv credit card is convenient for customers because they do not need to authenticate at the Fiserv site during the checkout flow. This way, consumers can stay in your experience and checkout faster and easier.

Consumers can store up to four Fiserv credit cards.

Follow these steps to allow consumers to add a Fiserv stored payment in your experience.

1. Call Get Stored Payments by UPMID to check if the stored payment exists. If so, skip the rest of this section and proceed to Payment Preview.
2. Make a POST call to the Start a Fiserv Bill Key Registration endpoint, passing the billing and shipping information, `returnURL` and `cancelURL` so Fiserv can return the consumer to your experience in **Step 3**. A successful response contains the Fiserv `url` and the `fields` object consisting of name/value pairs.
3. Make a POST call to the Fiserv `url`, passing each name/value pair from the `fields` object returned in **Step 2**. At the Fiserve site, the consumer completes their bill key registration. Once the consumer accepts or cancels registration, Fiserv redirects the consumer to either the `returnURL` and `cancelURL` you provided in **Step 2**, and appends their own query parameters needed to save the Fiserve stored payment at Nike.
4. Make a POST call to the Save a Fiserve Bill Key Registration endpoint to save the Fiserve bill key registration and payment details.
5. (Optional) Call Get Stored Payments by UPMID again to list all of the consumer's saved payments to make sure the newly added Fiserve credit card is in the list.

Listed below is a sample **Start a Fiserv Bill Key Registration** POST request URI. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

```
https://api.nike.com/commerce/storedpayments/payment/fiserv_billkeyreg/v1
```

A successful 200 response lists the Fiserve bill key registration `url` and the name/value pairs in the `fields` object for your experience to send in the Fiserve bill key registration POST call in **Step 3** above.

Listed below is a sample **Save a Fiserve Bill Key Registration** POST request URI in **Step 4** above. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

```
https://api.nike.com/commerce/storedpayments/payment/fiserv_savepayment/v1
```

A 201 response indicates a status of `success`. Other possible responses are a 500 and 429.

Credit Card Payment

Add Credit Card with CVV

Add Credit Card without CVV

Add or Update Credit Card CVV

Add or Update Credit Card Expiry and CVV

Validate Credit Card

Store Credit Card for Validation and Purchase

Get Credit Card

Get and Validate Credit Card

This service lists, modifies, deletes and stores a consumer's credit card and Apple Pay information. This service accommodates both PCI-certified and non-PCI-certified experiences. Endpoints for non-PCI-certified experiences render an iFrame to collect and retrieve credit card and Apple Pay information. Javascript on the IFrame validates and stores the credit card information once the consumer enters it, so your experience doesn't have to call those endpoints. For PCI-certified experiences, it offers endpoints to manage a consumer's credit card and Apple Pay information directly.

Add Credit Card with CVV

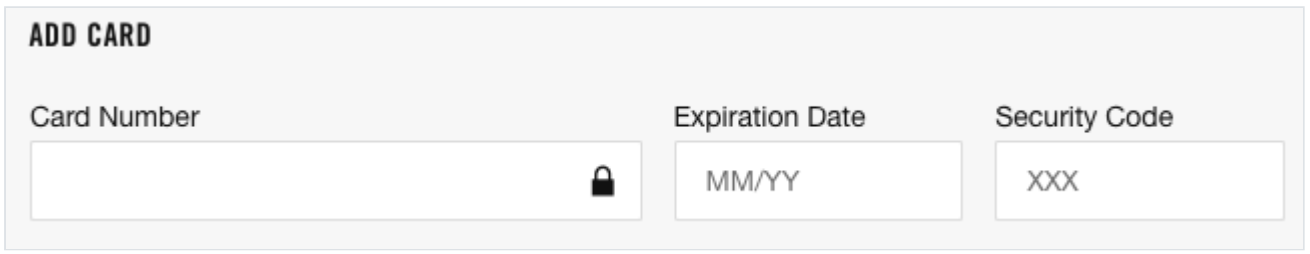
The Add Credit Card Info with CVV endpoint is called by experiences that are not PCI-certified. This endpoint renders an iFrame with editable masked credit card number, expiration date, and CVV fields pre-populated with values matching the `creditCardInfoId` passed in the path parameter. If the `creditCardInfoId` is not found, the iFrame renders blank, editable credit card number, expiration date and CVV fields. When each field has a value, the iFrame calls the Store Credit Card for Validation and Purchase endpoint and temporarily stores the new or updated values. As a last step, the iFrame calls the Validate Credit Card endpoint to validate the new or updated credit card data.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Listed below is a sample Add Credit Card Info with CVV GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services?id=24afd5dc-
b523-491c-8282-8bed57cd2029&ctx=checkout&language=en
```

The response renders the iFrame below with editable credit card number, expiration date, and CVV fields pre-populated with values looked up based on the creditCardInfoId {id} path parameter.



Add Credit Card without CVV

The Add Credit Card Info without CVV endpoint is called by experiences that are not PCI-certified. This endpoint renders an iFrame with the editable masked credit card number and expiration date fields matching the `creditCardInfoId` passed in the path parameter. If the `creditCardInfoId` is not found, the iFrame renders blank, editable credit card number and date fields. When each field has a value, the iFrame calls the Store Credit Card for Validation and Purchase endpoint and temporarily stores the new or updated values. As a last step, the iFrame calls the Validate Credit Card endpoint to validate the new or updated credit card data.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Listed below is a sample Add Credit Card Info without CVV GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services/add?id=0e13e71d-e952-46af-
b3f5-e476befd43ce&language=en
```

The response renders the iFrame below with editable credit card number and expiration date fields pre-populated with values looked up based on the `creditCardInfoId` path parameter.

Add or Update Credit Card CVV

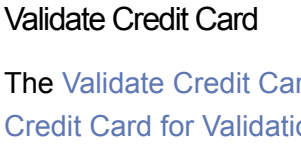
The Add or Update Credit Card CVV endpoint is called by experiences that are not PCI-certified. This endpoint renders an iFrame with an editable CVV field. When the consumer provides a CVV value, the iFrame calls the Store Credit Card for Validation and Purchase endpoint and temporarily stores the CVV if it found a credit card matching the `creditCardInfoId` path parameter. As a last step, the iFrame calls the Validate Credit Card endpoint to validate the updated CVV.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Listed below is a sample Add or Update Credit Card CVV GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services/cvv?id=0e13e71d-e952-46af-
b3f5-e476befd43ce&language=en
```

The response renders the iFrame below with an editable CVV field.



Add or Update Credit Card Expiry and CVV

The Add or Update Credit Card Expiry and CVV endpoint is called by experiences that are not PCI-certified. This endpoint renders an iFrame with an editable credit card expiration date and CVV fields. When the consumer provides the appropriate values, the iFrame calls the Store Credit Card for Validation and Purchase endpoint and temporarily stores the data for the `creditCardInfoId` path parameter. As a last step, the iFrame calls the Validate Credit Card endpoint to validate the updated expiration date and CVV values.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Listed below is a sample Add or Update Credit Card Expiry and CVV GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/services/expcvv?id=0e13e71d-e952-46af-
b3f5-e476befd43ce&language=en
```

The response renders the iFrame below with editable expiration date and CVV fields pre-populated with values looked up by the `creditCardInfoId` path parameter.

Validate Credit Card

The Validate Credit Card endpoint is typically called by the credit card payment iFrames immediately after the Store Credit Card for Validation and Purchase endpoint to validate credit card information. The mode query parameter determines which credit card values to validate. It can be called directly by PCI-cerfified experiences.

- 1: validates expiration date, credit card number and CVV
- 2: validates expiration date and credit card number
- 3: validates CVV
- 4: validates expiration date and CVV

Listed below is a sample Validate Credit Card GET request URI. It is not JWT-restricted.

```
https://payment.nike.com/creditcardsubmit/24afd5dc-b523-491c-8282-8bed57cd2029/
isValid?mode=3
```

Each field in the response body is flagged either true or false. True indicates the value is valid; false indicates invalid.

Sample Validate Credit Card response body for mode=1:

```
{
  "exp": true,
  "cc": true,
  "cvv": true,
  "isValid": true
}
```

Sample Validate Credit Card response body for mode=2:

```
{
  "exp": true,
  "cc": true,
  "isValid": true
}
```

Sample Validate Credit Card response body for mode=3:

```
{
  "cvv": true,
  "isValid": true
}
```

Sample Validate Credit Card response body for mode=4:

```
{
  "exp": true,
  "cvv": true,
  "isValid": true
}
```

Store Credit Card for Validation and Purchase

The Store Credit Card for Validation and Purchase endpoint temporarily stores credit card information for validation and purchase. This endpoint is called by the credit card payment iFrames. It can be called directly by PCI-certified experiences.

Listed below is a sample Store Credit Card for Validation and Purchase POST request URI. It is not JWT-restricted.

```
https://payment.nike.com/creditcardsubmit/bb3360c5-82c3-4d00-b806-a8bf3875555
```

A successful response is a 201.

Get Credit Card

The Get Credit Card endpoint retrieves masked credit card information for a `creditCardInfoId`. It can be called directly by PCI-certified experiences.

Listed below is a Get Credit Card GET request URI. It is not JWT-restricted.

```
https://paymentcc.nike.com/creditcardsubmit/bb3360c5-82c3-4d00-b806-a8bf3875555
```

A successful 200 response returns credit card information matching the `creditCardInfoId` passed in the path parameter.

Get and Validate Credit Card

The Get and Validate Credit Card endpoint retrieves masked credit card information and validation status of individual credit card fields for a `creditCardInfoId`. It can be called directly by PCI-certified experiences.

Listed below is a Get and Validate Credit Card GET request URI. It is not JWT-restricted.

```
https://paymentcc.nike.com/creditcardsubmit/bb3360c5-82c3-4d00-b806-a8bf3875555/
isValidData?mode=3
```

A successful 200 response returns the `isValid` flag for the credit card as a whole, and `isValid` flags for the credit card number and CVV. It also returns credit card information with a masked credit card number.

Apple Pay Payment

Start an Apple Pay Session

We recommend reading Apple Pay on the Web and Apple Pay JS API first.

Step 1: Check That the Consumer Can Pay By Apple Pay

To enable paying with Apple Pay on a Safari Web browser, your experience will need to call the Apple Pay JS API to validate that the Nike consumer can pay by Apple Pay on the web and get a `validationURL` to provide merchant identification to Apple.

In order to be eligible to pay by Apple Pay on a Safari web browser, the consumer must have:

- Access to a Mac and either an iPhone, iWatch, or iPad
- Installed the latest macOS Sierra or higher on Mac
- Installed iOS 10 or higher on the iPhone, iWatch, or iPad
- Set up Apple Pay on the iPhone, iWatch, or iPad
- Logged into the same iCloud account on Mac as iPhone, iWatch, or iPad
- Installed the latest version of Safari on iPhone, iWatch, or iPad

Step 2: Start an Apple Pay Session

Use the Start Apple Pay Session endpoint to initialize an Apple Pay session through the Apple gateway. Your experience passes the `validationURL` from **Step 1** to the endpoint, and the service will provide the necessary information to Apple Pay to identify Nike as a merchant that accepts Apple Pay payments and start a new Apple Pay session.

Step 3: Store Credit Card for Validation and Purchase

Once you get a successful 200 response from Start an Apple Pay Session, you have all the information you need to call Store Credit Card for Validation and Purchase and continue the purchase flow as you would for a credit card.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Listed below is a sample Start Apple Pay Session POST request URI and body. The `validationURL` is passed to your experience from the Apple Pay JS API when you provide merchant validation.

```
https://api.nike.com/payment/applepay_sessions/v2

{
  "validationURL":"https://apple-pay-gateway-pr-pod1.apple.com/paymentservices/
startSession"
}
```

A successful 200 response contains the encrypted `signature` that your experience needs to pass in the `paymentData` field to Store Credit Card for Validation and Purchase.

[Add credit card steps to complete Apple Pay payment]

Wallet Payment

Start a PayPal Express session

Start a PayPal Mark session

Get PayPal details including shipping and billing addresses stored at PayPal

Your experience can offer two, different PayPal flows, Express and Mark. What's the difference? See the table below.

Table 4: Comparison of PayPal Express and Mark Flows

Express Flow	Mark Flow
Shipping address is stored at PayPal Payment is made in the Nike Experience	Shipping address is stored in the Nike experience Payment is made at the PayPal site

PayPal Express

Follow these steps to implement the PayPal Express payment flow to your experience.

Step 1: Request PayPal Express

Before a consumer can pay in the PayPal Express flow, your experience must initiate an Express session at PayPal.

v2 Checkout

For v2 Checkout, use the Request PayPal Express v1 endpoint:

```
POST https://api.nike.com/payment/paypal_express/v1
```

v3 Checkout

For v3 Checkout, use the Request PayPal Express v2 endpoint:

```
POST https://api.nike.com/payment/paypal_express/v2
```

There are differences in the request body format between v1 and v2 (see the respective API Reference docs for details), but the flow is the same: you pass Checkout information, `returnURL` and `cancelURL` and these URLs are used by PayPal to return the consumer to your experience.

This service returns a `paypalToken` and `redirectURL` in the response. When the consumer is ready to pay, your experience redirects the consumer to the PayPal `redirectURL` passing the `paypalToken`. PayPal uses the token to look up the PayPal session.

At the PayPal site, the consumer either:

- Chooses the method of payment such as PayPal balance, debit card, or credit card. All payment methods saved at PayPal have a billing address associated with them.
- Chooses an existing or adds a new shipping address
- Confirms the payment method and shipping address

OR

- Cancels the PayPal Express session

When the consumer confirms the payment method and shipping address on the PayPal site, PayPal redirects the consumer to the `returnURL` passed in the request body. The `returnURL` is typically to a Checkout review page in your experience from which the consumer can choose to submit the Checkout for fulfillment.

If the consumer cancels the PayPal Express session on the PayPal site, PayPal redirects the consumer to the `cancelURL` passed in the request body. The `cancelURL` is typically to a billing page in your experience where the consumer can choose an alternate payment method.

> **TIP**: The `returnURL` you provide in the Request PayPal Express **request body** is a link to your experience to which PayPal will redirect the consumer after confirming the payment method and shipping address. The `returnURL` provided in the Request PayPal Express **response body** is a link to the PayPal site to which your experience will redirect the consumer to select a shipping address and payment method.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the Using Nike APIs guide to learn more about working with asynchronous Nike APIs.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling ETA. A successful response in `COMPLETED` status also includes the response object containing the job results.

Step 2: Retrieve PayPal Express Job

Use this endpoint to check the status of the PayPal Express job. After receiving a HTTP 202 and waiting the duration of the ETA time, call the endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

v2 Checkout

For v2 Checkout, use the Retrieve PayPal Express Job v1 endpoint:

```
GET https://api.nike.com/payment/paypal_express/v1/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

v3 Checkout

For v3 Checkout, use the Retrieve PayPal Express Job v2 endpoint:

```
GET https://api.nike.com/payment/paypal_express/v2/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

To know if the job is done, check the value of the status field in the response body as follows:

- "status": "PENDING": job processing has not started
- "status": "IN_PROGRESS": job processing in progress
- "status": "COMPLETED": job has completed

Once you receive a job status of "COMPLETED", get the results of your job by parsing the data in the response object from this endpoint.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the Request PayPal Express response.

A successful 200 response lists the `paypalToken` and `redirectURL`

Step 3: Request PayPal Details

Use this endpoint to retrieve and validate PayPal data, including shipping and billing addresses stored at PayPal. You will need to pass the `paypalToken` and `shoppingCountry` in the request body, as returned in either the Request PayPal Mark or Request PayPal Express response.

v2 Checkout

For v2 Checkout, use the Request PayPal Details v1 endpoint:

```
POST https://api.nike.com/payment/paypal_details/v1
```

v3 Checkout

For v3 Checkout, use the Request PayPal Details v2 endpoint:

```
POST https://api.nike.com/payment/paypal_details/v2
```

Common Considerations

- This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.
- This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the Using Nike APIs guide to learn more about working with asynchronous Nike APIs.
- A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling ETA. A successful response in `COMPLETED` status also includes the response object containing the job results.
- When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Step 4: Retrieve PayPal Details Job

Use this endpoint to check the status of the PayPal Details job. After receiving a HTTP 202 and waiting the duration of the ETA time, call the endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

v2 Checkout

For v2 Checkout, use the Retrieve PayPal Express Job v1 endpoint:

```
GET https://api.nike.com/payment/paypal_details/v1/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

v3 Checkout

For v3 Checkout, use the Retrieve PayPal Express Job v2 endpoint:

```
GET https://api.nike.com/payment/paypal_details/v2/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

To know if the job is done, check the value of the status field in the response body as follows:

- "status": "PENDING": job processing has not started

- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the Request PayPal Express response.

A successful 200 response lists shipping and billing addresses.

PayPal Mark

Follow these steps to implement the PayPal Mark payment flow to your experience.

Step 1: Request PayPal Mark

Before a consumer can pay in the PayPal Mark flow, your experience must initiate a Mark session at PayPal.

v2 Checkout

For v2 Checkout, use the Request PayPal Mark v1 endpoint:

`POST https://api.nike.com/payment/paypal_mark/v1`

v3 Checkout

For v3 Checkout, use the Request PayPal Mark v2 endpoint:

`POST https://api.nike.com/payment/paypal_mark/v2`

Common v1/v2 Considerations

You pass in Checkout information, shipping address, `returnURL` and `cancelURL` to initiate the Mark session. These URLs are used by PayPal to return the consumer to your experience.

This endpoint starts a new PayPal Mark session and passes the Checkout information and shipping address to PayPal for session storage. PayPal generates a `paypalToken` and `redirectURL` and the endpoint returns them in the response. When the consumer is ready to Pay, your experience redirects the consumer to the PayPal `redirectURL` passing the `paypalToken`. PayPal uses the token to look up the PayPal session.

At the PayPal site, the consumer either:

- chooses the method of payment such as PayPal balance, debit card, or credit card. All payment methods saved at PayPal have a billing address associated with them.
- confirms payment and pays for the Checkout

OR

- cancels the PayPal Mark session

After the consumer pays for the Checkout or cancels the PayPal Mark session on the PayPal site, PayPal redirects the consumer to either the `returnURL` or `cancelURL` passed in the request body. The `returnURL` is typically to an order confirmation page in your experience.

> **TIP**: The `returnURL` you provide in the Request PayPal Mark **request body** is a link to your experience to which PayPal will redirect after the consumer pays for their Checkout at the PayPal site. The `returnURL` provided in the Request PayPal Mark **response body** is a link to PayPal to which your experience will redirect the consumer to pay.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the Using Nike APIs guide to learn more about working with asynchronous Nike APIs.

> **TIP**: When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling ETA. A successful response in `COMPLETED` status also includes the response object containing the job results.

Step 2: Retrieve PayPal Mark Job

Use this endpoint to check the status of the PayPal Mark job. After receiving a HTTP 202 and waiting the duration of the ETA time, call the endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

v2 Checkout

For v2 Checkout, use the Retrieve PayPal Mark Job v1 endpoint:

```
GET https://api.nike.com/payment/paypal_mark/v1/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

v3 Checkout

For v3 Checkout, use the Retrieve PayPal Mark Job v2 endpoint:

```
GET https://api.nike.com/payment/paypal_mark/v2/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the Request PayPal Express response.

A successful 200 response lists the `paypalToken` and `redirectURL`

Step 3: Request PayPal Details (PayPal Mark)

For details, see Request PayPal Details in the PayPal Express section.

Step 4: Retrieve PayPal Job (PayPal Mark)

For details, see Retrieve PayPal Details Job in the PayPal Express section.

Deferred Payment

Generate a signed link to pay at third-party vendor sites such as iDeal, Sofort, Alipay, Tenpay and UnionPay

Generate a signed link to pay by WeChat

Get the status of a deferred payment

Step 1: Request Deferred Payment Form

Use the Request Deferred Payment Form endpoint to generate a signed link used to redirect the consumer to pay at a third-party site or app. This endpoint is used for experiences that support these payment types:

- iDeal
- Sofort and/or Alipay
- Tenpay
- UnionPay

For WeChat payment, see the Request WeChat Deferred Payment endpoint.

In the request body, your experience will need to pass the `approvalId` returned from Request Payment Approval and your experience's `returnURL` that the third-party vendor will redirect the consumer to after making payment at their site.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the Using Nike APIs guide to learn more about working with asynchronous Nike APIs.

Listed below is a sample Request Deferred Payment Form POST request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/deferred_payment_forms/v1
```

A successful 202 response in the `PENDING` or `IN_PROGRESS` status includes a link to the job and a status polling ETA. A successful response in `COMPLETED` status also includes the response object containing the job results.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Parsing the 'COMPLETED' job result directly is best practice because it eliminates making another service call.

Step 2: Retrieve Deferred Payment Form Job

Use the Retrieve Deferred Payment Form Job endpoint to check the status of the Request Deferred Payment Form job. After receiving a HTTP 202 and waiting the duration of the ETA time, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status. In the case of checking the status of Request WeChat Deferred Payment, see the Request WeChat Deferred Payment section.

To know if the job is done, check the value of the `status` field in the response body as follows:

- `"status": "PENDING"`: Job processing has not started
- `"status": "IN_PROGRESS"`: Job processing in progress
- `"status": "COMPLETED"`: Job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the `response` object from this endpoint.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the Deferred Payment Form response.

Listed below is a sample Retrieve Deferred Payment Form Job GET request URI. The endpoint is not JWT-restricted.

```
https://api.nike.com/payment/deferred_payment_forms/v1/jobs/
2722be3a-0341-11e6-b512-3e1d05defe78
```

A successful 200 response in `COMPLETED` status contains the signed third-party vendor URL at which the consumer can pay for their Nike order.

Step 3: Redirect the Consumer to the Third-Party Payment site

Use the values from the Request Deferred Payment Form job to redirect the consumer to pay at the third-party site. Depending upon the vendor and the experience the consumer is shopping in, the response may contain a URL to generate a QR code for the deferred payment page, or a form action URL and HTTP method.

Step 4: Request Deferred Payment Status

Use the Request Deferred Payment Status endpoint to check the status of a deferred payment with the third-party Vendor.

Your experience will need to pass the `approvalId` returned from Request Payment Approval and any `vendorData` returned in the Request Deferred Payment Form response.

This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the Using Nike APIs guide to learn more about working with asynchronous Nike APIs.

Listed below is a sample Request Deferred Payment Status POST request URI. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

```
https://api.nike.com/payment/deferred_payment_status/v1
```

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

A successful 202 response in `COMPLETED` status lists the payment status and amount paid if the payment status is `PAYMENT_SUCCESSFUL`. If the 202 response is `PENDING` or `IN_PROGRESS`, it includes a link to the job and a status polling ETA.

Step 5: Retrieve Deferred Payment Status Job

Use the Retrieve Deferred Payment Status Job endpoint to check the status of the Request Deferred Payment Status. After receiving a HTTP 202 and waiting the duration of the ETA time, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the Deferred Payment Status response.

Listed below is a sample Retrieve Deferred Payment Status Job GET request URI. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

```
https://api.nike.com/payment/deferred_payment_status/v1/jobs/
2722be3a-0341-11e6-b512-3e1d05defe78
```

A successful 200 response in `COMPLETED` status lists the payment status and amount paid if the payment status is `PAYMENT_SUCCESSFUL`.

WeChat Deferred Payment

We recommend reading JSAPI WeChat Browser first.

WeChat Desktop Flow

Consumers scan a QR code to pay with WeChat in a web browser flow, also known as native payment.

The Vendor generates a transaction QR Code according to the WeChat Payment Protocol, and the payer goes to “Scan QR Code” in their WeChat in order to complete payment. This mode is applicable to payments made on websites, physical stores, media advertising, or other scenarios.

Desktop flow: Experience displays QR code at the end of checkout (what generates the QR code?). Consumer scans code with phone and opens WeChat app. User pays. (when is wechat deferred payment called? how is nike notified of payment?

Step 1: Generate a QR Code

If the consumer is shopping in a desktop web experience and chooses to pay by WeChat, your experience will need to generate a QR code at the end of Checkout for the consumer to scan on a mobile phone. Generate the QR code by following the steps in WeChat's documentation on WeChat Login for WebApps.

Step 2: Request WeChat Deferred Payment

Use the Request WeChat Deferred Payment endpoint to generate the necessary values to initiate a session in the WeChat Pay Browser Phone App from a mobile or desktop web browser. For other third-party deferred payment types, see Request Deferred Payment Form.

The mobile web flow opens the WeChat Payment app directly when it is time to pay for the Nike Checkout.

The Desktop WeChat flow generates a QR code. Nike's consumers use their Mobile phone to scan the code to open the WeChat Payment App on their mobile device.

Using the code returned from WeChat in **Step 1** and the `approvalId` returned in the Request Payment Approval response, call Request WeChat Deferred Payment.

Listed below is a sample Request WeChat Deferred Payment POST request URI. This endpoint is not JWT-restricted.

```
https://api.nike.com/payment/deferred_wechat_payments/v1
```

A successful 202 response includes a link to the job and a status polling ETA.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Step 3: Retrieve WeChat Deferred Payment Job

Use the Retrieve WeChat Deferred Payment Job endpoint to check the status of the Request WeChat Deferred Payment job. After receiving a HTTP 202 and waiting the duration of the ETA time, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- "status": "PENDING": job processing has not started
- "status": "IN_PROGRESS": job processing in progress
- "status": "COMPLETED": job has completed

Once you receive a job status of "COMPLETED", get the results of your job by parsing the data in the response object from this endpoint.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the Deferred Payment Form response.

Listed below is a sample Retrieve Deferred WeChat Payment Job GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/deferred_wechat_payments/v1/jobs/
2722be3a-0341-11e6-b512-3e1d05defe78
```

Response Body

The HTTP 200 response from *Deferred WeChat Payment Job* contains information about how to retrieve the results of your job via the 'Deferred WeChat Payment Job' endpoint. The response body fields are the same as those returned in the WeChat Deferred Payment response except the resourceType is payment/deferred_wechat_payments/jobs.

WeChat Mobile Flow

When consumers pay by WeChat in a mobile phone web browser, your experience will open the WeChat app to process the payment, also known as In-App payment.

In-App payment refers to a mobile-based payment in which the Vendor calls the WeChat payment module by using the open SDK integrated in their mobile-based app to pay for transactions.

After consumer chooses to pay with wechat, experience calls WeChat deferred payment to get values. Calls JS API using those values to open WeChat App. User pays. WeChat sends callback to notify experience of payment. Experience loads order confirmation page.

Korea Payment

- Start and Save a Fiserv Billing Key Registration
- Ready a payment for vendor authentication
- Get the Ready Payment job result
- Call Vendor UI for authentication and payment information

Step 1: Register a Bill Key

Ready a payment for vendor authentication

For registered consumers who choose to pay by Fiserv credit card, your experience must first check if the consumer has saved the credit card as a stored payment. See Start and Save a Fiserv Billing Key Registration in **Storing Payment** for more information.

If the customer is paying with a stored payment Fiserv credit card, you can skip the rest of this section and proceed to Payment Preview where you will pass the stored payment `paymentId`.

Step 2: Ready a payment for vendor authentication

If the consumer is not paying with a stored payment credit card, you need to initiate a session with the vendor site for all Korea payment types including credit cards. This step gathers information needed by the vendor when the consumer visits their site to authenticate and provide payment details during the checkout flow. Do this by calling Request Ready Payment once the consumer has selected the Korea payment method in your experience.

The supported Korea Payment types are:

- KakaoPay
- Naver Pay
- Credit card
- PayCo
- Bank transfer

Pass `paymentType`, `checkoutId`, `returnURL`, `cancelURL`, `failURL` and order details such as shipping and billing information in the **Ready Payment** request. The vendor uses the `returnURL` to redirect the consumer after successful authentication and gathering of payment information, the `cancelURL` if the consumer cancels the action, and the `failURL` in case of an error.

> **TIP:** When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Listed below is a sample Request Ready Payment POST request URI. This endpoint is asynchronous.

```
https://api.nike.com/payment/ready_payment/v1
```

Response Body

The successful 202 response contains the UUID job `id` used to retrieve the job results in **Step 3** and the job status with a `resourceType` value of `payment/ready_payment`.

A response in the PENDING or IN_PROGRESS status includes a link to the job (including the UUID job id) and a status polling ETA.

A response in COMPLETED status also includes the response object containing the job results.

Step 3: Get the Ready Payment job result

Use the Retrieve Ready Payment Job endpoint to check the status of the Request Ready Payment. After receiving a HTTP 202 and waiting the duration of the ETA time, call this endpoint using the same UUID job `id` to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the Retrieve Ready Payment Job response.

Listed below is a sample Retrieve Ready Payment Job GET request URI with 621827cc-82b4-408b-9e63-7292795fa233 as the `id` path parameter.

```
https://api.nike.com/payment/ready_payment_jobs/v1/
621827cc-82b4-408b-9e63-7292795fa233
```

Response Body

Depending upon the `paymentType`, a successful 200 response includes a `fields` object containing an array of name/value pairs and may include the payment vendor `url`. Your experience passes the fields as query parameters in the vendor’s `url` so the consumer can authenticate at the vendor’s site, explained in the next step.

- **All Fiserv payments**: The response also contains an encrypted `signature` that your experience passes in the `paymentData` field to Payment Preview.
- **Fiserv credit card payments**: The response contains a hash number that you will pass in Step 4.
- **Naver Pay payments**: Use the values in the `fields` object in the response and pass them to the vendor’s script loaded in your UX. Note that vendor `url` is not returned in the Naver Pay response.

Step 4: Call Vendor UI for authentication and payment information

After initiating a session with the vendor site in Step 2 and gathering the job results in Step 3, it’s time for the consumer to authenticate with the payment vendor and provide their payment information. The consumer’s experience depends upon on the `paymentType` they select.

KakaoPay Web

After you receive the KakaoPay `url` and `fields` from Step 3, your web experience opens the KakaoPay `url` passing the name/value pairs from the `fields` object as query parameters, including the unique KakaoPay transaction ID (TID). KakaoPay uses the TID to link transactions together such as approvals and cancellations.

KakaoPay authentication flow:

- Your experience loads the KakaoPay payment request page in a layer or popup appending the name/value pairs in the `fields` object as query parameters
- On the KakaoPay payment request page, the consumer either scans the QR code on the web browser from their phone or sends themselves a payment message through the KakaoTalk App
- In the KakaoTalk app, the consumer selects the payment method and completes authentication
- Once authentication is complete, the KakaoPay payment request page redirects to one of three urls:
  - **Success**: If payment is successful, KakaoPay redirects the consumer to the `returnURL` you provided in **Step 2**, appending the authorization `pg_token` as a query parameter. This token is required for payment approval.
  - **Cancel**: If the consumer decides to cancel during authentication, KakaoPay redirects the user to the `cancelURL` you provided in **Step 2**
  - **Fail**: If payment is not completed within 15 minutes of calling initiating a ready payment request, KakaoPay redirects the consumer to the `failURL` you provided in **Step 2** and the transaction is cancelled
- Your experience calls payment preview passing the KakaoPay `pg_token` in the `authorizationToken` field.
- Nike checkout flow continues normally, including authorizing the KakaoPay payment through Checkouts
- Once the consumer completes Nike checkout, the consumer receives a confirmation push notification and email from KakaoPay as well as an order confirmation email from Nike

> **TIP**: Scroll to the bottom of the KakaoPay Integration Confluence page to view the KakaoPay Developer guide for more information.

KakaoPay Mobile

Similar to the KakaoPay Web flow, your mobile app displays the `url` from Step 3 in a webview.

This opens the KakaoTalk mobile app where the consumer selects the payment method and completes authentication on the KakaoTalk payment page.

From here, the experience is identical to the KakaoPay Web flow.

Naver Pay

Naver Pay provides a simple version of their script to both display the Naver Pay button in your experience and load their payment form UI, which uses the standard Naver Pay button. You can create your own button using the custom version of the script.

> **Note**: Naver Pay does not allow loading their payment form in an iFrame for security reasons.

Your experience can open the Naver Pay payment form by these **Open Type** methods. If you do not want to open the form by the default method, use the **custom** version of the script:

Open Type	Web	Mobile
`layer`	X (default)	
`page`	X	X (default)
`popup`	X	X

An example of how to load the **simple** Naver Pay script is displayed below:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body><!--// mode : development or production-->
<!--// data-chain-id : For group type, enter the chainIdvalue.-->
<script src="https://nsp.pay.naver.com/sdk/js/naverpay.min.js"
  data-client-id="{#_clientId}"data-mode="{#_mode}"
  data-merchant-user-key="{#_merchantUserKey}"
  data-merchant-pay-key="{#_merchantPayKey}"
  data-product-name="{#_productName}"
  data-total-pay-amount="{#_totalPayAmount}"
  data-tax-scope-amount="{#_taxScopeAmount}"data-tax-ex-scope-
amount="{#_taxExScopeAmount}"
  data-return-url="{#_returnUrl}">
</script>
</body>
</html>
```

An example of how to load the **custom** Naver Pay script is displayed below:

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
<input type="button" id="naverPayBtn" value="NAVER Pay button">
<script src="https://nsp.pay.naver.com/sdk/js/naverpay.min.js"></script>
<script>var oPay = Naver.Pay.create({ // See the SDK parameters."mode" :
"{#_mode}","clientId": "{#_clientId}"//"chainId" : "{For grouptype, enter the chainId
value.}"});// Assign a click event on the custom NAVER Pay button.var elNaverPayBtn =
document.getElementById("naverPayBtn");elNaverPayBtn.addEventListener("click",
function(){oPay.open({ // See the Pay Reserve parameters."merchantUserKey":
"{#_merchantUserKey}","merchantPayKey": "{#_merchantPayKey}","productName":
"{#_productName}","totalPayAmount": {#_totalPayAmount},"taxScopeAmount":
{#_taxScopeAmount},"taxExScopeAmount": {#_taxExScopeAmount},"returnUrl":
"{#_returnUrl}"});});</script>
</body>
</html
```

Naver Pay authentication flow:

- Naver Pay script displays the Naver Pay payment form in the Open Type your experience defined so the consumer can authenticate with Naver Pay
- In the Naver Pay payment form, the consumer logs in, selects escrow or pay with card, and agrees to share their payment information with nike.com
- When Naver Pay requires self-verification, the consumer enters their birthday and phone number
  - Naver Pay system calls the consumer's phone number with a verification code
  - Consumer enters the verification code in the Naver Pay UI
- Once authentication is complete, Naver Pay redirects the consumer to the `redirectURL` passed in **Step 2** with the Naver Pay-generated `PaymentId` and `resultCode` (`Success` or `Fail`) query parameters appended
- Your experience calls Payment Preview, passing the Naver Pay `PaymentId` in the `authorizationToken` field
- Nike checkout flow continues normally, including authorizing the Naver Pay payment through Checkouts
- Once the consumer completes Nike checkout, the Consumer receives a confirmation push notification and email from Naver Pay as well as an order confirmation email from Nike

> **TIP**: Scroll to the bottom of the Naver Pay Integration Confluence page to view the Naver Pay Integration guide for more information.

Credit Card, Payco, and Bank Transfer

These three payment methods go through the Fiserv Korea Payment Gateway and have identical consumer flows. Fiserv offers a hosted payment page that your UX experience loads from the Checkout Payment page.

Fiserv authentication flow:

- Your experience redirects to the Fiserv `url` from Step 3, passing the name/value pairs from the `fields` object
- On the Fiserv site, the consumer selects the payment method and completes authentication
  - If paying by credit card, the consumer provides their credit card information
  - If paying by PayCo, Fiserve redirects the consumer to complete authentication at the PayCo site
- Once authentication is complete, Fiserv redirects the consumer to the `returnURL` you provided in Step 2, appending the authorization `FDTid` as a query parameter. This token is required for payment approval.
- Your experience calls payment preview passing the Fiserv `FDTid` in the `authorizationToken` field
- Nike checkout flow continues normally, including authorizing the Fiserv payment through Checkouts

> **TIP**: Scroll to the bottom of the Fiserv Credit Card Integration Confluence page to view the Fiserv UI Integration guide for more information.

Payment Preview

Preview the allocation of payment amounts across one or more payment methods selected by the consumer

Step 1: Request a Payment Preview

Nike's consumers can pay by one or more gift cards and vouchers and another payment type such as PayPal or credit card. This endpoint allocates payment to the gift cards or voucher with the highest balance first, then to the rest of the gift cards and vouchers on the Checkout in ascending balance order. If the total balance of all gift cards and vouchers is less than the order amount, the service allocates the balance of the order to a second payment type.

v2 Checkout

For v2 Checkout, use the Payment Preview v2 endpoint:

```
POST https://api.nike.com/payment/preview/v2
```

OR

```
PUT https://api.nike.com/payment/preview/v2/2722be3a-0341-11e6-b512-3e1d05defe783424
```

v3 Checkout

For v3 Checkout, use the Request Payment Preview v3 endpoint:

```
POST https://api.nike.com/payment/preview/v3
```

OR

```
PUT https://api.nike.com/payment/preview/v3/2722be3a-0341-11e6-b512-3e1d05defe783424
```

Common Considerations

- The `paymentPreviewId` returned by this service is a required key when calling Request Checkout Submit to validate and authorize/debit payment before submitting a Checkout to Nike for fulfillment.
- This endpoint operates **asynchronously** which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the Using Nike APIs guide to learn more about working with asynchronous Nike APIs.
- When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
- `Promotion` in the response.payments.`type` field is an indicator that the entire order is allocated to a promotion.
- A successful 202 response includes a link to the job and a status polling ETA.

More About the Payment Preview Request Body:

- `checkoutId` is a UUID that payments are associated to. It is generated by Request Checkout Preview.
- items.shippingAddress.`county` holds the shipping address county for the US. Outside of the US, it holds regional data and is required in CN and JP.
- `paymentInfo` is an array of payment types for the Checkout and is required except for the PayPal Mark flow.
- paymentInfo.`billingInfo` is required for all payment methods except PayPal.
- PaymentInfo.`creditCardInfoId` is a required field when paying by credit card that is not a stored payment. It is a PCI-required token used to look up credit card information and is generated by the Store Credit Card for Validation and Purchase endpoint. If the credit card is a stored payment, then PaymentInfo.**paymentId** is required.
- SMS Payment Preview v3 allows both Nike members and guests to purchase Nike products using a mobile phone number instead of an email address. To implement SMS in Payment Preview v3, pass the SMS phone number in `billing.contactInfo.phoneNumber` and leave `billing.contactInfo.email` null.

Note: SMS Payment Preview v3 is currently available in China only

Step 2: Check That the Payment Preview Job has Finished

Use this endpoint to check the status of the Payment Preview job. After receiving a HTTP 202 and waiting the duration of the ETA time, call the endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

v2 Checkout

For v2 Checkout, use the Retrieve Paymen Preview Job v2 endpoint:

```
GET https://api.nike.com/payment/preview/v2/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

v3 Checkout

For v3 Checkout, use the Retrieve Payment Preview Job v3 endpoint:

```
GET https://api.nike.com/payment/preview/v3/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint. Alternatively, follow the link to the Retrieve Payment Preview Result endpoint which is provided in the `links` object response body.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` field from the Request Payment Preview response.
> - Parsing the “COMPLETED” job result directly is best practice because it eliminates making another service call.

Step 3: Retrieve Payment Preview Result

After calling the Request Payment Preview to start the job and Retrieve Payment Preview Job to check that the status of the job is “COMPLETED”, you can optionally call the Retrieve Payment Preview Result endpoint to retrieve the job result. It is optional because the result is also returned in the Retrieve Payment Preview Job when it is in “COMPLETED” status.

Listed below is a sample Retrieve Payment Preview Result GET request URI. It is not JWT-restricted.

v2 Checkout

```
GET https://api.nike.com/payment/preview_results/v2/308830db-bcca-45a6-8d81-20f3b6dafd9e
```

v3 Checkout

```
GET https://api.nike.com/payment/preview_results/v3/308830db-bcca-45a6-8d81-20f3b6dafd9e
```

A successful 200 response lists the payment types on the Checkout, and the amount allocated to each type.

> **TIPS:**
>
> - When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.
> - Get the {id} path parameter from the `id` job UUID in the *Request Payment Preview* response.

3-D Secure Authentication

Prevent fraud and protect the consumer

The Payment 3DS service (3-Domain Secure) adds a layer of protection against fraud in credit card and debit card transactions. It uses Adyen, a third-party 3D Secure 2 provider, to authenticate payment transactions. Not all credit card payment transactions require 3DS. If 3DS is required, your experience calls the Payment 3DS service as a separate step before payment authorization, which takes place during Request Checkout Submit.

When do I need to call this service?

If `is3DSRequired` is **true** in the Payment Preview response, the transaction requires 3DS authentication. In this case, your experience needs to call the Request Authentication endpoint in the Payment 3DS API. Depending upon the response, you will either call additional 3DS and Adyen endpoints or proceed directly to Request Checkout Submit.

> **TIP:** All 3DS POST endpoints are synchronous.

Step 1: Request Authentication

Call the Request Authentication endpoint passing the `paymentPreviewId` from the Request Payment Preview response, currency, originURL, returnURL, channel, amount, and browser information.

Listed below is a sample Request Authentication POST request URI. **This endpoint is JWT-restricted.**

```
/payment/3ds_authentications/v1
```

A successful response includes a `resultCode` that determines the authentication flow. Check the table below to learn what steps you need to take next.

Table 5: Result Codes with Next Steps

Result Code	Next Step
AuthenticationFinished	The payment was successfully authenticated with 3DS 2 and no further calls to the 3DS API are required. Proceed to **Step 5: Request Checkout Submit**.
IdentifyShopper	The consumer's device fingerprint is required in order to authenticate the payment with 3DS 2. Proceed to **Step 2: Request Fingerprint**.
ChallengeShopper	The consumer must complete an authentication challenge in order to authenticate the payment with 3DS 2. Proceed to **Step 3: Request Challenge**.
RedirectShopper	The transaction could not be authenticated using 3DS 2. Redirect the consumer to the issuer's site to authenticate the transaction using 3DS 1. Proceed to **Step 4: Redirect Shopper**.
Error	An error occurred during the call. Display the error to the consumer.

Step 2: Request Fingerprint

If the Request Authentication call returned a resultCode of **IdentifyShopper**, you will need to get the secure device fingerprint.

Step 2a: Get the Fingerprint Result Token

Follow Adyen's fingerprint flow for Web, iOs or Android in your app or experience passing the `token` as the fingerprintToken from the Request Authentication response in **Step 1**. A successful response returns the device fingerprint result token.

Step 2b: Get the Fingerprint

Once you have the device fingerprint result token from **Step 2a**, call the Request Fingerprint endpoint passing the `paymentPreviewId` from the Request Payment Preview response, and the device fingerprint result token.

Listed below is a sample Request Fingerprint POST request URI. **This endpoint is JWT-restricted.**

```
/payment/3ds_fingerprint_shoppers/v1
```

A successful response includes a `resultCode` that determines the authentication flow. Check the table below to learn what steps you need to take next.

Table 6: Result Codes with Next Steps

Result Code	Next Step
AuthenticationFinished	The payment was successfully authenticated with 3DS 2 and no further calls to the 3DS API are required. Proceed to **Step 5: Request Checkout Submit**.
ChallengeShopper	The consumer must complete an authentication challenge in order to authenticate the payment with 3DS 2. Proceed to **Step 3: Request Challenge**.
Error	An error occurred requesting the fingerprint. Display the error to the consumer.

Step 3: Request Challenge

If the Request Fingerprint call returned a resultCode of **ChallengeShopper**, you will need to present an authentication challenge to the consumer.

Step 3a: Present a Challenge

Follow Adyen's present a challenge flow for Web, iOs or Android in your app or experience passing the `token` from the Request Fingerprint response from **Step 2** as the challenge token. A successful response returns a challenge result token.

Step 3b: Request Challenge

Once you have the challenge result token from **Step 3a**, call the Request Challenge endpoint passing the `paymentPreviewId` from the Request Payment Preview response, and the challenge result token as the `challengeResultToken`.

Listed below is a sample Request Challenge POST request URI. **This endpoint is JWT-restricted.**

```
/payment/3ds_challenge_shoppers/v1
```

A successful response includes a `resultCode` that determines the authentication flow. Check the table below to learn what steps you need to take next.

Table 7: Result Codes with Next Steps

Result Code	Next Step
AuthenticationFinished	The consumer was successfully authenticated with 3DS 2 and no further calls to the 3DS API are required. Proceed to **Step 5: Request Checkout Submit**.
Error	An error occurred requesting the fingerprint. Display the error to the consumer.

Step 4: Redirect Shopper

If the Request Authentication call returned a resultCode of **RedirectShopper**, you will need to redirect the consumer to the issuer's site to authenticate the transaction using 3DS 1 as a fallback.

Step 4a: Redirect the Consumer to the Issuer's Site

Redirect the consumer to the `url` from the Request Authentication response, so the consumer can complete payment authentication. Once the payment is successfully authenticated at the bank site, the consumer will be redirected to your site with `MD` and `PaRes` variables appended.

Step 4b: Request Redirect

After the transaction was successfully authenticated at the issuer's site in **Step 4a**, call the Request Redirect endpoint passing the `paymentPreviewId` from the Request Payment Preview response and `MD` and `PaRes` URL parameters returned in **Step 4a**.

Listed below is a sample Request Redirect POST request URI. **This endpoint is JWT-restricted.**

```
/payment/3ds_redirect_shoppers/v1
```

A successful response includes a `resultCode` that determines the authentication flow. Check the table below to learn what steps you need to take next.

Table 8: Result Codes with Next Steps

Result Code	Next Step
AuthenticationFinished	The consumer was successfully authenticated with 3DS 1 and no further calls to the 3DS API are required. Proceed to **Step 5: Request Checkout Submit**.
Error	An error occurred requesting the fingerprint. Display the error to the consumer.

Step 5: Request Checkout Submit

Once the 3DS transaction has been authenticated, follow the steps for Request Checkout Submit when the consumer is ready to complete the purchase.

Payment Approval

Perform fraud check, validation, and authorization/debit for all payment types on a consumer's Checkout

Void a Payment Approval request

Get a Payment Approval summary

> **TIP:** Except for the Get Payment Approval Summary endpoint, only service-to-service calls should be made to the Payment Approval endpoints. Request Checkout Submit calls Request Payment Approval (POST) as a last step in the Checkout flow to validate and authorize/debit payment before submitting a Checkout to Nike for fulfillment. **An experience should not call this service directly**.

Step 1: Request Payment Approval

This endpoint performs fraud check, validation, and authorization/debit for all payment types on a consumer's Checkout.

v2 Checkout

For v2 Checkout, use the Request Payment Approval v2 endpoint:

```
POST https://api.nike.com/payment/approval/v2
```

OR

```
PUT https://api.nike.com/payment/approval/v2/2722be3a-0341-11e6-b512-3e1d05defe783424
```

v3 Checkout

For v3 Checkout, use the Request Payment Approval v3 endpoint:

```
POST https://api.nike.com/payment/approval/v3
```

OR

```
PUT https://api.nike.com/payment/approval/v3/2722be3a-0341-11e6-b512-3e1d05defe783424
```

Common Considerations

- The PUT version of the endpoint requires a `paymentApprovalId` path parameter. This is helpful if the Request Payment Approval response times out. In that case, Request Payment Void would need to be called with the same `paymentApprovalId` to reverse the original Payment Approval request.
- This endpoint (POST or PUT) must be called after Request Checkout Preview so that order allocation is calculated and the `paymentPreviewId` is assigned.
- The Payment Approval service validates the payment allocation performed by the Payment Preview service, recalculating if necessary, and evaluates that the selected payment methods and items on Checkout are valid. If one or more payment type validations fail, all gift card debits and all credit card and PayPal authorizations are rolled back. This service uses the `paymentPreviewId` to look up the Checkout payment methods, so it does not require payment information be passed in the request.
- This endpoint operates **asynchronously**, which means that there are extra steps to retrieve the results of your request. Read the Asynchronous Operation section of the Using Nike APIs guide to learn more about working with asynchronous Nike APIs.
- A successful 202 response includes a link to the job and a status polling ETA.

SMS Support (China only)

SMS Payment Approval v3 allows both Nike members and guests to purchase Nike products using a mobile phone number instead of an email address. If the consumer is purchasing using an email address, you can skip this section.

> Note: SMS checkout is currently available in China only

In addition to the usual Payment Approval request values, these are SMS-specific:

- Send the value 'SMS_ACCOUNT' in `phoneNumber.type`
- For Nike members, send the SMS phone number from the Nike members's profile in `phoneNumber.subscriberNumber`, 1 - 13 digits. For guest users, send the mobile number they provide.
- Send the country code in `phoneNumber.countryCode`, 1 - 3 digits
- For Nike members, send the Nike member's profile ID in `phoneNumber.accountId`
- For guest users, send the validation token acquired in SMS Checkout Preview in `phonenumber.verifyId`

Step 2: Retrieve Payment Approval Job

Use this endpoint to check the status of the Request Payment Approval (POST) or Request Payment Approval (PUT) job. After receiving a HTTP 202 and waiting the duration of the ETA time specified in the response, call Retrieve Payment Approval Job using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

v2 Checkout

For v2 Checkout, use the Retrieve Payment Approval Job v2 endpoint:

```
GET https://api.nike.com/payment/approval/v2/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

v3 Checkout

For v3 Checkout, use the Retrieve Payment Approval Job v3 endpoint:

```
GET https://api.nike.com/payment/approval/v3/jobs/2722be3a-0341-11e6-b512-3e1d05defe783424
```

To know if the job is done, check the value of the status field in the response body as follows:

- “status”: “PENDING”: job processing has not started
- “status”: “IN_PROGRESS”: job processing in progress
- “status”: “COMPLETED”: job has completed

Once you receive a job status of “COMPLETED”, get the results of your job by parsing the data in the response object from this endpoint.

A successful 200 response gives the job status. If `COMPLETED`, the response lists the Payment Approval job results.

Step 3: Retrieve Payment Approval Result (Optional)

After calling Request Payment Approval (POST) or Request Payment Approval (PUT) to start the job and Retrieve Payment Approval Job to verify the status of the job is “COMPLETED”, you can optionally call Retrieve Payment Approval Results to retrieve the result of your Payment Approval job. This step is optional because the Payment Approval result is also returned in the Retrieve Payment Approval Job when it is in `COMPLETED` status. DOMS is currently the only service that calls this endpoint.

v2 Checkout

For v2 Checkout, use the Retrieve Payment Approval Result v2 endpoint:

GET `https://api.nike.com/payment/approval_results/v2/ae6575a7-8c0e-44ef-b91b-440bdaf2070b`

v3 Checkout

For v3 Checkout, use the Retrieve Payment Approval Result v2 endpoint:

GET `https://api.nike.com/payment/approval_results/v3/ae6575a7-8c0e-44ef-b91b-440bdaf2070b`

A successful 200 response lists the Payment Approval job results.

Step 4: Request Payment Void (Optional)

Use the Request Payment Void endpoint to void a Request Payment Approval. This endpoint should be called when the original Request Payment Approval (PUT) timed out using the same {id} sent in the path parameter. If a credit card or PayPal was used in the original Payment Approval request, this endpoint reverses the authorization. If a gift card was used in the original Payment Approval request, it reverses the debit.

v2 Checkout

For v2 Checkout, use the Request Payment Void v2 endpoint:

DELETE `https://api.nike.com/payment/approval_results/v2/ae6575a7-8c0e-44ef-b91b-440bdaf2070b`

v3 Checkout

For v3 Checkout, use the Request Payment Void v3 endpoint:

DELETE `https://api.nike.com/payment/approval_results/v3/ae6575a7-8c0e-44ef-b91b-440bdaf2070b`

A successful response is a 204.

Step 5: Get Payment Approval Summary (Optional)

> **TIP**: This endpoint can be called directly by an experience to display the masked results of a Request Payment Approval in `COMPLETED` status.

Use the Get Payment Approval Summary endpoint to retrieve a summary of a successful Payment Approval request. The results are available for 30 minutes after the original Payment Approval request was made. This endpoint can be called by an experience because sensitive account information is masked. The results can be used to display payment approval results confirming a consumer's order.

Note that if the Payment Approval result is not in either `ACCEPT` or `PENDING_PAYMENT` status, the service returns a 404 response.

> **TIP**: When calling this endpoint through the public router, the `upmid` (for logged-in consumers), `appId` and `usertype` headers are automatically added by the Nike Edge Router based on the access token in the Authorization header populated by accounts.nike.com or Nike Unite/Identity.

Listed below is a sample Get Payment Approval Summary GET request URI. It is not JWT-restricted.

```
https://api.nike.com/payment/approval_summary/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response lists a summary of a successful payment approval with masked account information.

Post Order Payment Processing

The following payment actions can be taken on an order after it has been submitted for fulfillment.

Debit an account

Credit an account

Void a payment authorization

Create an electronic gift certificate

Create an electronic voucher

Get payment status

Generate a CyberSource report

Nike's Distributed Order Management System (DOMS) is currently the only consumer of this service.

All Payment Gateway endpoints are **asynchronous**. After making the initial request to a Payment Gateway endpoint, you will need to poll the appropriate job endpoint to get the results.

All PUT requests in the Payment Gateway API require a service-to-service JWT in the `X-Nike-Authorization` header both to identify the calling service and to prove that the calling service is authorized to call the endpoint.

> **TIP**: The {id} path parameter for PUT requests to this service is a client-supplied value used to look up the job result.

Debit an Account

Step 1: Submit the Debit Request

Use the Request Debit endpoint to debit funds for orders paid by credit card, PayPal, Gift Certificate, and Klarna payment types. This endpoint is called when it is time to transfer funds from the consumer's account to Nike's account, such as when the consumer's shipment leaves the warehouse.

Listed below is a sample Request Debit PUT request URI. **This endpoint requires a service-to-service JWT in the X-Nike-Authorization header**.

```
/payment/debits/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

> **TIP**: The `authorizationRequestId` and `authorizationRequestToken` request body fields come from the `requestId` and `requestToken` returned in the Request Payment Approval response.

A successful 202 response includes a link to the job and a status polling ETA.

Step 2: Check the Debit Job Status

Use the Retrieve Debit Job endpoint to retrieve the results of the Request Debit job. After receiving a HTTP 202 and waiting the duration of the ETA time specified in the response, call this endpoint using the same UUID to check the status of your job. If the status is not COMPLETED, continue the cycle of waiting the ETA period and checking the job status.

To know if the job is done, check the value of the `status` field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample Retrieve Debit Job GET request URI. This endpoint is not JWT-restricted.

```
/payment/debits/v1/jobs/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Debit job results.

Credit an Account

Step 1: Submit the Credit Request

When a Nike consumer returns one or more products, call the Request Credit endpoint to return the money to the same account the consumer used to pay for the product. You can issue a full credit (refund) for the full charge amount, or you can issue multiple, partial credits up to the full charge amount. If you try to credit more than the charge amount, you will receive an error.

> **TIP**: You can get the `debitRequestId` and `debitRequestToken` values to pass in the credit request body from the `requestId` and `requestToken` fields in the Retrieve Debit Job.

Listed below is a sample Request Credit PUT request URI. **This endpoint requires a service-to-service JWT in the X-Nike-Authorization header**.

```
/payment/credits/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling ETA.

Step 2: Check the Credit Job Status

After calling Request Credit and receiving a HTTP 202 response, execute a request to Retrieve Credit Job using the same Credit ID to check the status of your job.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample Retrieve Credit Job GET request URI. This endpoint is not JWT-restricted.

```
/payment/credits/v1/jobs/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Credit job results.

Void an Authorization

Step 1: Request an Unauth

Use the Request Unauth endpoint to release the hold on funds set aside by authorization for a future debit. See the Payment Gateway API for details on void request and response information for each payment type.

> **TIP**: The `authorizationRequestId` and `authorizationRequestToken` request body fields come from the `requestId` and `requestToken` in the Request Payment Approval response.

Listed below is a sample Request Unauth PUT request URI. **This endpoint requires a service-to-service JWT in the X-Nike-Authorization header**.

```
/payment/authorize_reversals/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling ETA.

Step 2: Retrieve the Unauth Job

Use the Retrieve Unauth Job endpoint to retrieve the status of the Request Unauth job. After receiving a HTTP 202 and waiting the duration of the ETA time specified in the response, call this endpoint using the same UUID to check the status of your job. If the status is not `COMPLETED`, continue the cycle of waiting the ETA period and checking the job status.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample Retrieve Unauth Job GET request URI. This endpoint is not JWT-restricted.

```
/payment/authorize_reversals/v1/jobs/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Unauth job results.

Reauthorize a Payment

There is no need to reauthorize a payment after the authorization has expired. The Payment Gateway will handle that for you based on the authorization expiration date.

Create an Electronic Gift Certificate

Step 1: Submit Payment Request for an Electronic Gift Certificate

You can create an electronic gift certificate for a consumer by calling the Create Gift Certificate endpoint. Pass the sender and recipient information, currency and amount in the request body.

Listed below is a sample Create Gift Certificate PUT request URI. **This endpoint requires a service-to-service JWT in the X-Nike-Authorization header**.

```
/payment/gift_certificates/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling ETA.

Step 2: Check the Request Certificate Job Status

After calling Create Gift Certificate and receiving a HTTP 202 response, execute a request to Retrieve Gift Certificate Create Job Status using the same {id} to check the status of your job. When the job has completed successfully, this endpoint returns an expiration date, gift certificate number, and PIN set to the amount and currency passed in the job request.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample Retrieve Gift Certificate Create Job Status GET request URI. It is not JWT-restricted.

```
/payment/gift_certificate_jobs/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Retrieve Gift Certificate Create Job Status results.

Create an Electronic Voucher

Step 1: Submit Payment Request for Voucher

You can create an electronic voucher for a consumer by calling the Create Voucher endpoint. Pass the sender and recipient information, currency and amount in the request body.

Listed below is a sample Create Voucher PUT request URI. **This endpoint requires a service-to-service JWT in the X-Nike-Authorization header**.

```
/payment/vouchers/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling ETA.

Step 2: Check the Create Voucher Job Status

After calling Create Voucher and receiving a HTTP 202 response, execute a request to Retrieve voucher create job status using the same {id} to check the status of your job.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample Retrieve voucher create job status GET request URI. It is not JWT-restricted.

```
/payment/voucher_jobs/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Create Voucher job results including the voucher account number, amount, and currency passed in the job request.

Query Payment Status

Step 1: Submit the Payment Status Query

Use the Request Payment Status Query to check the payment status of an order. Before cancelling an order paid by a deferred payment type such as WeChat or Alipay, you will need to know if the order has been PAID so you can Request Credit to return the funds to the consumer’s account.

> **TIP**: The `requestId` and `requestToken` request body fields come from the `requestId` and `requestToken` in the Request Payment Approval response.

Listed below is a sample Request Payment Status Query PUT request URI. **This endpoint requires a service-to-service JWT in the X-Nike-Authorization header**.

```
/payment/queries/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 202 response includes a link to the job and a status polling ETA.

Step 2: Check the Retrieve Payment Status Query Job Status

After calling Request Payment Status Query and receiving a HTTP 202 response, execute a request to Retrieve Payment Status Query using the same {id} to check the status of your job.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample Retrieve Payment Status Query GET request URI. This endpoint requires a service-to-service JWT in the X-Nike-Authorization header.

```
/payment/queries/v1/jobs/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Request Payment Status job results including the status of the payment, order ID, amount, and currency.

Generate a CyberSource Report

You can generate a CyberSource report that details the payment status of Nike orders for your organization. The report lists orders that were processed through CyberSource, a third-party payment processing and fraud management system. It contains information such as order number, transaction date, settlement amount, and settlement date.

Step 1: Generate a CyberSource Report

Kick off a CyberSource report for your organization using the Retrieve CyberSource Report endpoint. Provide your organization ID (i.e. merchant account), report name, and the requested report date. You do not need to send a report ID in the request. The service generates the ID for you and returns it in the response.

Listed below is a sample Retrieve CyberSource Report POST URI request. **This endpoint requires a service-to-service JWT in the X-Nike-Authorization header**.

```
/payment/cybersource_reports/v1
```

A successful 202 response includes a link to the job, job status, job status polling ETA, and a report ID.

Step 2: Retrieve the CyberSource Report

After calling Retrieve CyberSource Report and receiving a HTTP 202 response, execute a request to Retrieve CyberSource Report Job using the job link in the response from **Step 1** to check the status of your job.

To know if the job is done, check the value of the status field in the response body as follows:

“status”: “PENDING”: job processing has not started

“status”: “IN_PROGRESS”: job processing in progress

“status”: “COMPLETED”: job has completed

Once you receive a job status of COMPLETED, get the results of your job by parsing the data in the response object from this endpoint.

Listed below is a sample Retrieve CyberSource Report Job GET request URI. It is not JWT-restricted.

```
/payment/cybersource_report_jobs/v1/ae6575a7-8c0e-44ef-b91b-440bdaf2070b
```

A successful 200 response gives the job status. If COMPLETED, the response lists the Retrieve CyberSource Report job results that include a signed link to S3 where you can view and download the report.

Third-Party Payment Notification

Third-party payment vendors call the Payment Notification API to notify Nike of payment status changes for an order with an offline payment type.

When consumers choose to pay for their Nike order with an offline payment type, consumers pay after the order is submitted for fulfillment. The consumer must pay within a certain time period defined by the vendor or Nike automatically cancels the order. After the consumer pays the third-party vendor, the vendor calls the Payment Notification service to notify Nike of payment. This service handles updating the payment status and making sure that DOMS is notified of the payment event, so it can update the order status.

The supported third-party payment vendors are:

- Alipay
- Konbini
- Sofort
- Unionpay
- WeChat

Each vendor has its own synchronous endpoint with the POST body defined by the vendor. The service response is either `success`, or `failure` and reason. See the Payment Notification API for details on request and response by vendor.

API Quick Reference

Payment ApplePay

- Start Apple Pay Payment Session

Payment Approval

v2:

- Request Payment Approval (POST)
- Request Payment Approval (PUT)
- Retrieve Payment Approval Job
- Retrieve Payment Approval Results
- Request Payment Void
- Get Payment Approval Summary

v3:

- Request Payment Approval (POST)
- Request Payment Approval (PUT)
- Retrieve Payment Approval Job
- Retrieve Payment Approval Results
- Request Payment Void

Credit Card Payment

- Add Credit Card with CVV
- Add Credit Card without CVV
- Add or Update Credit Card CVV
- Add or Update Credit Card Expiry and CVV
- Validate Credit Card
- Store Credit Card for Validation and Purchase
- Get Credit Card (with validation)
- Get and Validate Credit Card (without validation)

Deferred Payment

- Request Deferred Payment Form
- Retrieve Deferred Payment Form Job
- Request Deferred Payment Status
- Retrieve Deferred Payment Status Job
- Request WeChat Deferred Payment
- Retrieve WeChat Deferred Payment Job

Payment Gateway

- Request Debit
- Retrieve Debit Job
- Request Credit
- Retrieve Credit Job
- Request Unauth
- Retrieve Unauth Job
- Request Payment Status Query
- Retrieve Payment Status Query
- Create Gift Certificate
- Retrieve Create Gift Certificate Job
- Create Voucher
- Retrieve Voucher Create Job

- Retrieve CyberSource Report
- Retrieve CyberSource Report Job

Payment Options

v2:

- Get Payment Options
- Get Billing Countries for Shipping Country
- Validate Payments

v3:

- Get Payment Options

Payment Preview

v2:

- Request Payment Preview
- Retrieve Payment Preview Job
- Retrieve Payment Results

v3:

- Request Payment Preview (POST)
- Request Payment Preview (PUT)
- Retrieve Payment Preview Job
- Retrieve Payment Results

Stored Payment

- Get Stored Payments by UPMID
- Get Stored Payments by UPMID (Retail)
- Get Stored Gift Certificates by ID
- Add Stored Payment
- Delete Stored Payments by UPMID
- Modify Default Stored Payment
- Modify Credit Card Stored Payment
- Get Stored Payment by ID
- Delete Stored Payment by ID
- Validate Stored Payment Credit Card CVV
- Start a PayPal Billing Agreement
- Start a FiServ Billkey Registration V1
- Save a FiServ Billkey Registration V1

Payment Wallet

v2:

- Request PayPal Details
- Retrieve PayPal Details Job
- Request PayPal Express
- Retrieve PayPal Express Job
- Request PayPal Mark
- Retrieve PayPal Mark Job

v3:

- Request PayPal Details
- Retrieve PayPal Details Job
- Request PayPal Express
- Retrieve PayPal Express Job
- Request PayPal Mark
- Retrieve PayPal Mark Job

Korea Payment

- Ready Payment
- Retrieve Ready Payment Job

Caching Data

The Payment API makes use of data caching to optimize service SLAs. The first time data is fetched or when the cache expires, the Payment service makes a call to get the latest data and adds it to the cache.

The PaymentWallet, PaymentPreview, PaymentApproval and StoredPayments services handle gift card balances. Retrieving the balance of a gift card requires a call to a third-party gift card provider, which can slow down the Payment service's response, especially in high volume traffic. To avoid this scenario, the private gift card Service, which is responsible for retrieving gift card data and is called by the PaymentWallet, PaymentPreview, PaymentApproval and StoredPayments services, caches the gift card balance after retrieval. The cache time varies based on the balance. If the gift card has a positive balance, the gift card service caches the balance for 5 minutes; If the gift card has a 0 balance, the gift card service caches the balance for 30 minutes.

The PaymentOptions, PaymentWallet, PaymentPreview and PaymentApproval services use product and SKU data as part of validation. For performance reasons, these services cache product and SKU data for 30 minutes in order to reduce the amount of calls to the Merchandised Products API to get the latest data.

Best Practices

Sample Flows

Payment API flows vary based on the payment method, and the user experience. Listed below are a few examples of Payment API usage.

> **NOTE**: Although the flows refer to endpoints compatible with v2 Checkout, they also apply to endpoints compatible with v3 Checkout.

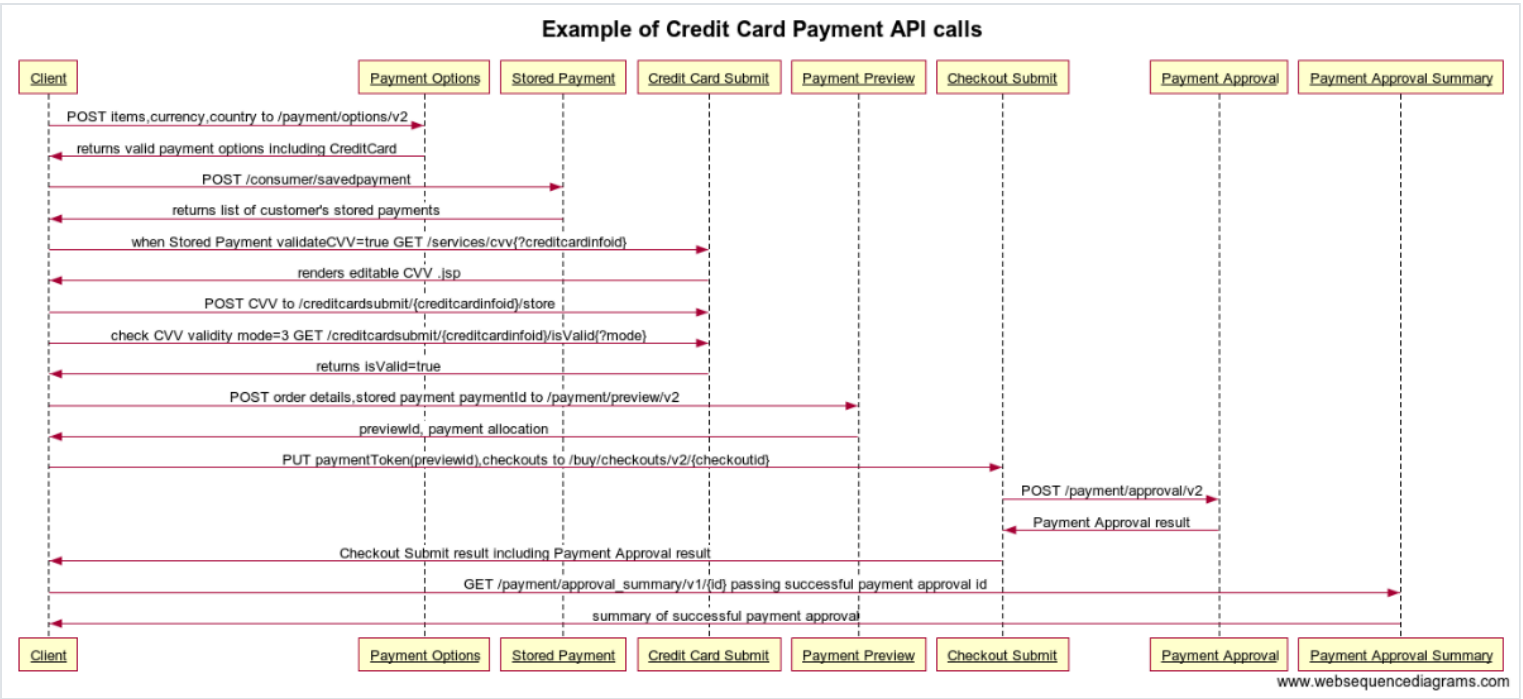
Sample Credit Card Payment Flow without Stored Payment

Listed below is a sample credit card payment flow. In this flow, the consumer is a Nike registered member who has added products and services to the Checkout, provided a shipping address, and has the intention to purchase.

1. Your experience calls Payment Options to get a list of valid payment methods for the consumer.
2. The consumer selects to pay by a non-stored credit card from the list of payment options in your app.
3. Your experience calls Credit Card Payment to capture the consumer's credit card information in a PCI-compliant UI. Since the consumer is a registered member, your experience may ask to store her credit card for future use.
4. If the registered member has chosen to store the credit card for reuse, your experience calls Stored Payment to validate, securely store, and display masked credit card information.
5. Your experience passes the Checkout and credit card information to Payment Preview in order to allocate the order total across the selected payment methods and generate the Payment Preview ID.
6. Your experience calls Checkout Preview to validate Checkout and calculate item pricing, shipping and taxes.
7. Your experience calls Checkout Submit with the Payment Preview ID to validate payment for a final time, authorizes the credit card by calling Payment Approval, and submits the Checkout for fulfillment.
8. Your experience calls Payment Approval Summary to display the payment details to the consumer for order confirmation.

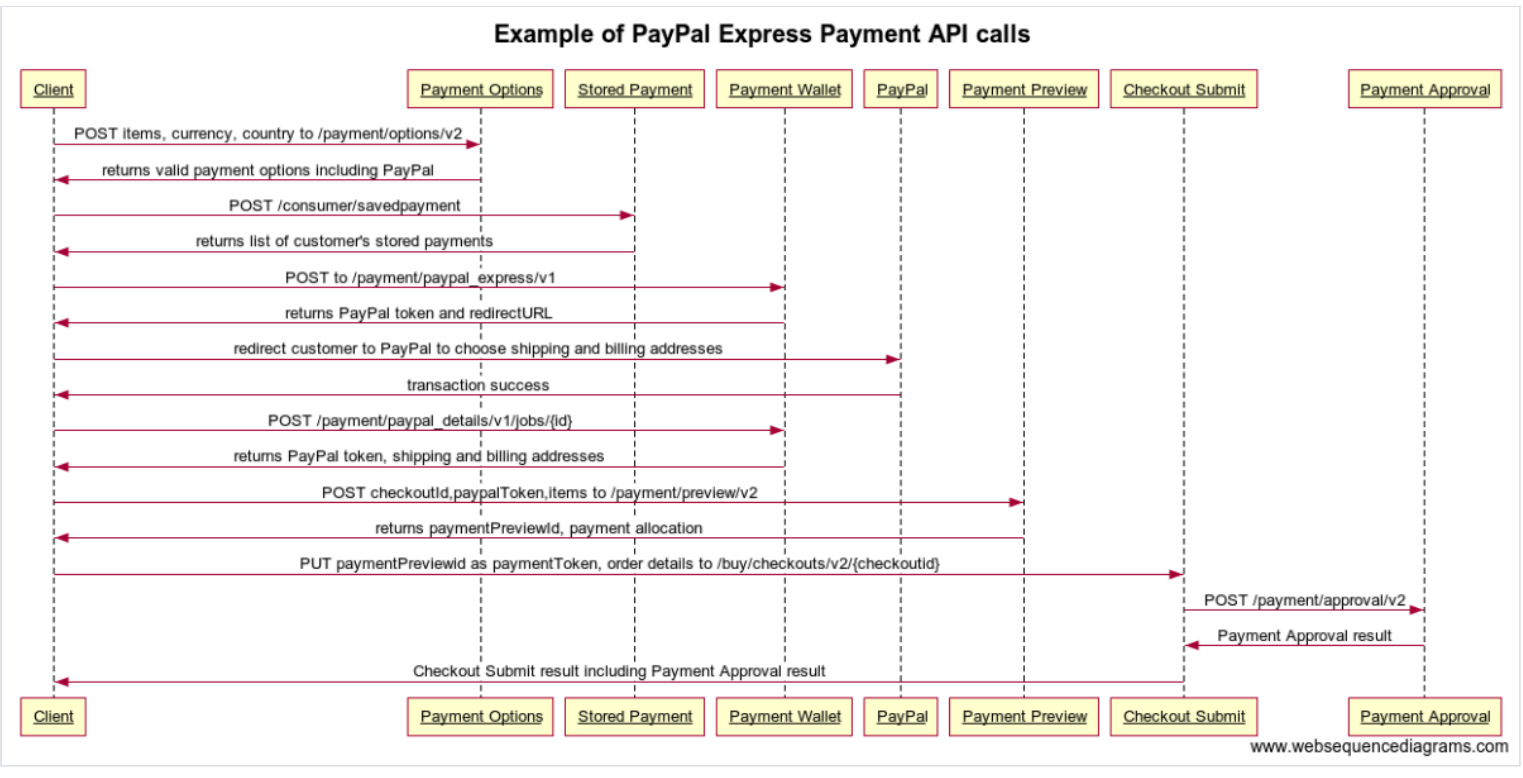
Sample Credit Card Payment Flow with Stored Payment

In this flow, the consumer chooses to pay by Credit Card that is saved as a stored payment method. The consumer must provide the CVV for validation because the shipping address passed into the call is either new or different from previous shipping addresses on past orders.



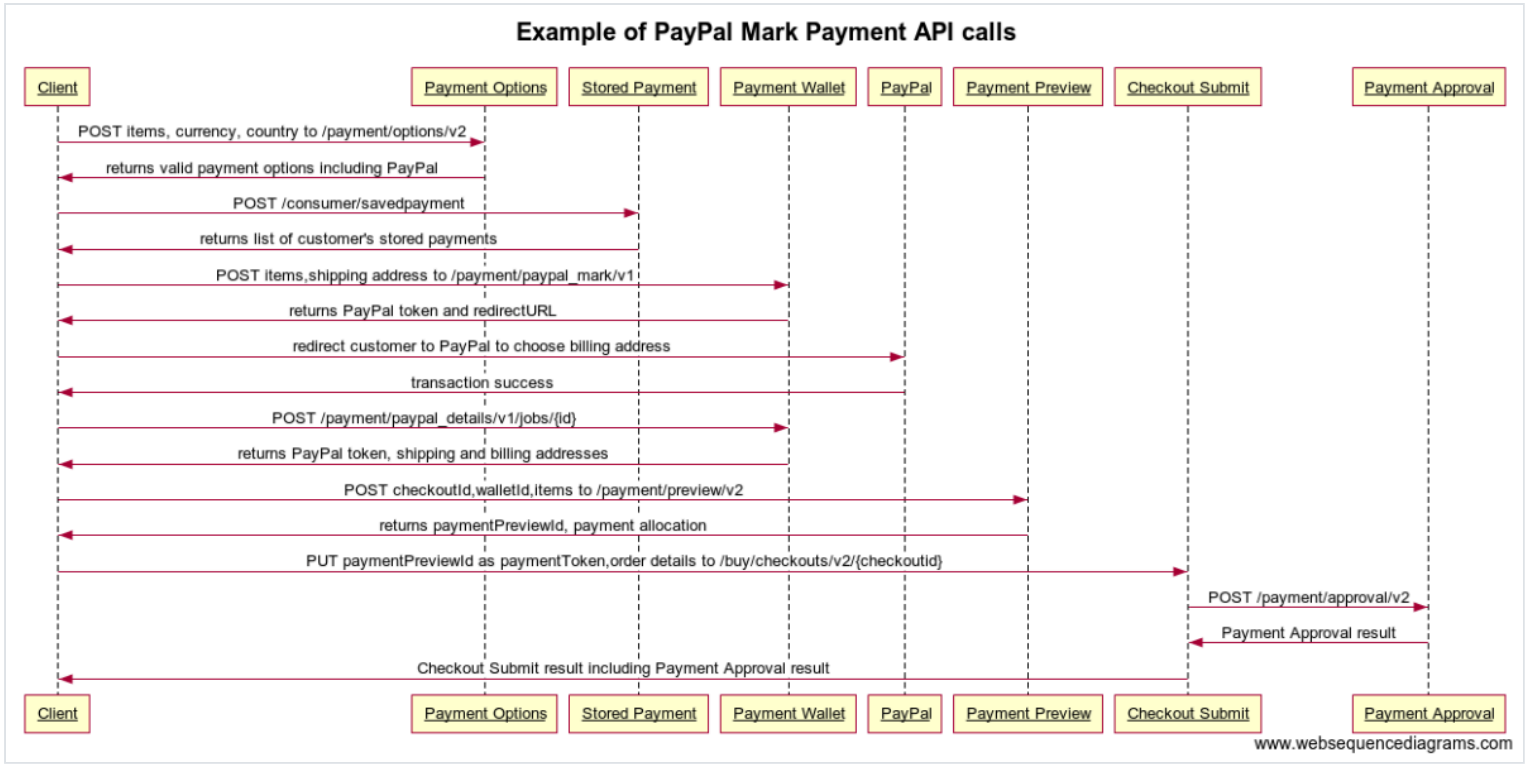
Sample PayPal Express flow

In this flow, the consumer is redirected to the PayPal site after choosing to pay by PayPal Express in the Nike experience. The consumer selects the shipping and billing addresses on the PayPal site. Based on the PayPal token, the Payment Wallet service returns the shipping and billing addresses from PayPal for display on the order confirmation.



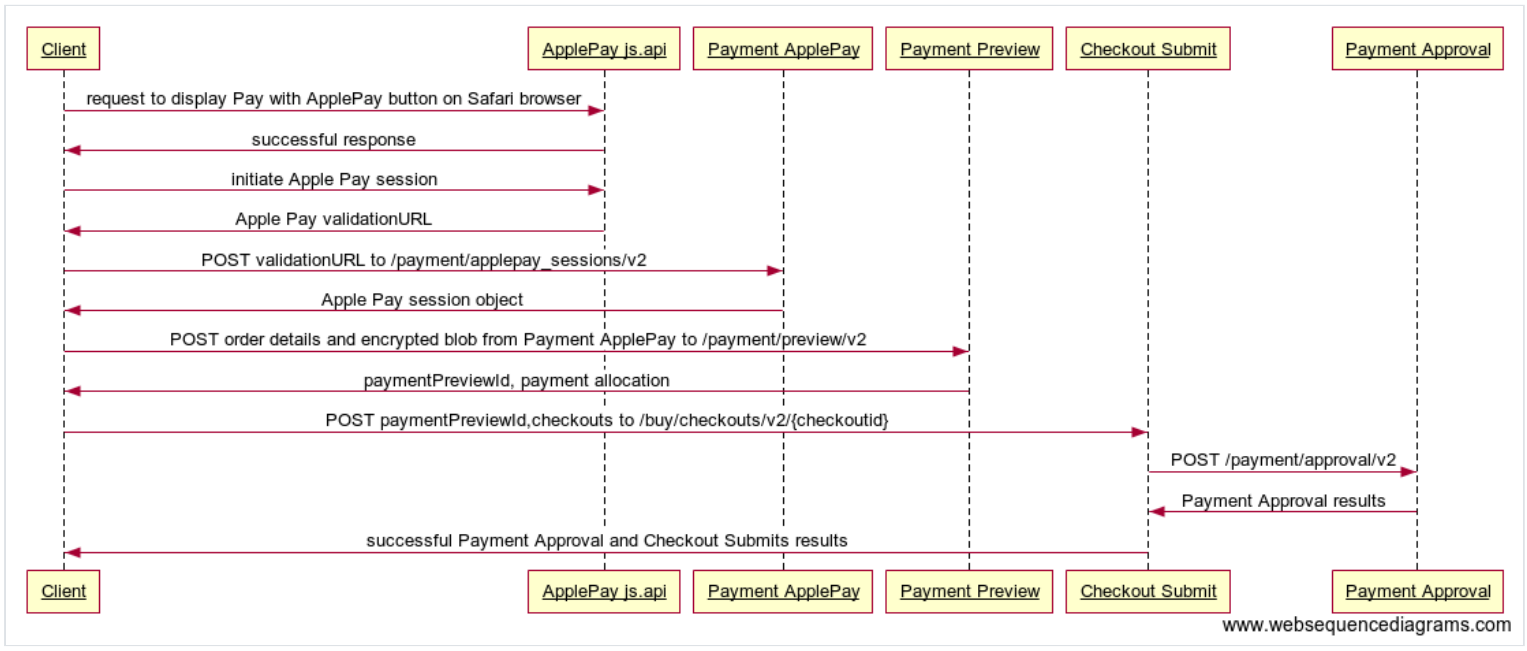
Sample PayPal Mark Flow

In this flow, the consumer chooses to pay by PayPal Mark and provides the shipping address in the Nike experience. From order review, the consumer is redirected to the PayPal site to select the billing address and pay. Based on the PayPal token, the Payment Wallet service returns the shipping and billing addresses from PayPal for display on the order confirmation.



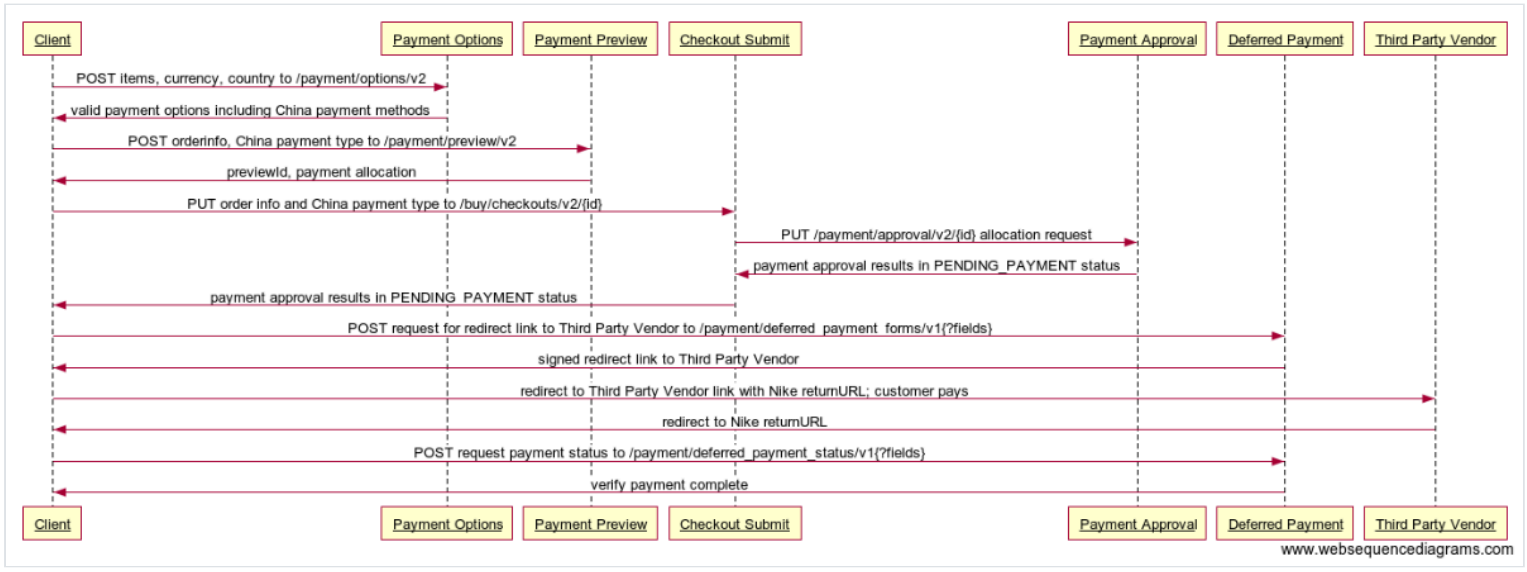
Sample Apple Pay Flow

In the example Payment API flow below, the consumer chooses to pay by Apple Pay in a Safari web browser.



Sample Deferred Payment Flow

In this flow, the consumer chooses to pay by a payment method that will be authorized and captured after the Nike order has been placed. This is a typical flow for China payment methods such as WeChat and Alipay.



Polling

To avoid excessive job polling of asynchronous endpoints, wait the number of milliseconds returned in the job request ETA before checking the job status.

Retry Conditions

For all Payment APIs, the general rule is that requests resulting in a HTTP 4XX response should not be retried without modification to the request data, but HTTP 5XX errors can be retried as is. For general information on Nike error retry practices, see API Error Patterns on Confluence.

The exception to the 4xx response rule is the 429 response, indicating that there are too many requests coming in for the service to handle. When a service returns a 429, the call should be retried a maximum of two times, using the value returned in the `Retry-After` header to determine when to make the follow-up call.

Troubleshooting

Listed below are some techniques to troubleshoot problems using the Payment API.

Query Splunk With a Trace ID

In order to abide by PCI-compliance rules, payment logging requires special Splunk access. As a result, you cannot query Splunk by Trace ID as a trouble-shooting tool to track down why a payment request failed. If need help from the Payment Team, post your problem to the #cic-payment Slack channel with details such as:

- Experience in which you encountered the error, iOS SNKRS app, nike.com web, Android SNKRS app, direct endpoint call etc.
- Time request failed
- Trace ID
- Request URI and body (if not GET request)
- Error codes and error messages

Inspect Browser Activity in a Live Experience

Try using your browser's built-in tools for inspecting web service calls made from a live Nike experience such as SNKRS Web. Or, set up Charles and your favorite device to proxy service calls made from SNKRS or other Nike apps. Sometimes seeing what other experiences are doing might address your question or concern.

> **TIP:** While inspecting http://www.nike.com/launch, you can change your shopping country with the flag icon at the upper right of the homepage to test different locales. Place orders in different countries with different payment methods to view the Payment call flow with other CiC services. Orders can be cancelled via self-service within 30 minutes of submission, otherwise contact Nike Consumer Services.

Terms of Service

It is recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of the Using Nike APIs guide on how to create and register your caller ID.

Authentication

Access Tokens

Most calls through the Nike API gateway (api.nike.com) require an access token be sent in the request header. This allows Nike to verify that your experience is authorized to perform the action on behalf of the user.

Access tokens are obtained by calling accounts.nike.com or Nike Unite/Identity prior to calling the API which you ultimately want to reach.

To find out more on how to call accounts.nike.com or Nike Unite/Identity to obtain access tokens, see the Authorization section of the Using Nike APIs guide.

JSON Web Token

A few of the endpoints in the Payment APIs require the additional authorization of a JSON Web Token (JWT). For more, see the JWT section of Using Nike APIs.

Contacting the Team

Need to contact the Payment team?

Slack	#cic-payment
Confluence Space	CiC Payment
Product Owner	Sree Krishna

Document Change Log

Date	Summary
Initial publish	01/8/2018
Added Payment Gateway detail	11/26/2018
Restructured for use cases	02/21/2018
Added Key Terms section	07/18/2019
Added voucher, gift certificate, and CyberSource report to Fulfillment section	10/21/2019
Added 3-D Secure Authentication section	11/14/2019
Added Source-Aware endpoints for Options, Wallet, Preview, Approval	05/7/2020
Added SMS support and third-party payment gateway (Adyen)	10/19/2021
Added Korea payment	03/8/2022
Added accounts.nike.com	01/04/2023

Next Steps

You've now learned how to add payment to your experience. Here are some next steps.

- Adding Order History to your experience
- Using Nike APIs
- Glossary